Artificial Intelligence (CS 1360)
CSP: Constraint Satisfaction Problems

This slide from: ai.berkeley.edu with slight modifications.

Outline

- **Introduction: Constraint Satisfaction Problem (CSP)**
  - **Backtrack (BT) algorithm**
    - Uninformed
    - Informed
    - **Improving BT:**
      - Most constrained variable
      - Most Constraining Variable
      - Least Constraining Value
  - **Constraints propagation (Arc Consistency)**
    - AC-3
    - Forward Checking

Introduction

- From the point of view of the search algorithm, each state is atomic, or indivisible—**a black box** with no internal structure.

- In this chapter, we use a **factored representation** for each state: a set of variables, each of which has a value.
  - A problem is solved when each variable has a value that satisfies all the constraints on the variable.
  - A problem described this way is called a **constraint satisfaction problem**, or CSP ➔ take advantage of the structure of states.
  - Eliminate large portions of the search space all at once by identifying variable/value combinations that violate the constraints.

Introduction

A constraint satisfaction problem consists of three components, X,D, and C:

- X is a set of variables, {X1, . . . ,Xn}.
- D is a set of domains, {D1, . . . ,Dn}, one for each variable.
- C is a set of constraints that specify allowable combinations of values.

Introduction

- **Constraint**: relationship allowing some situations and forbids others
  - i.e., a relation between multiple variables which limits the values these variables can take simultaneously.
- **Knowledge** can naturally be represented by **constraints**.
- **Reasoning**: constraint-based reasoning

Introduction

Map-coloring problem example as a CSP

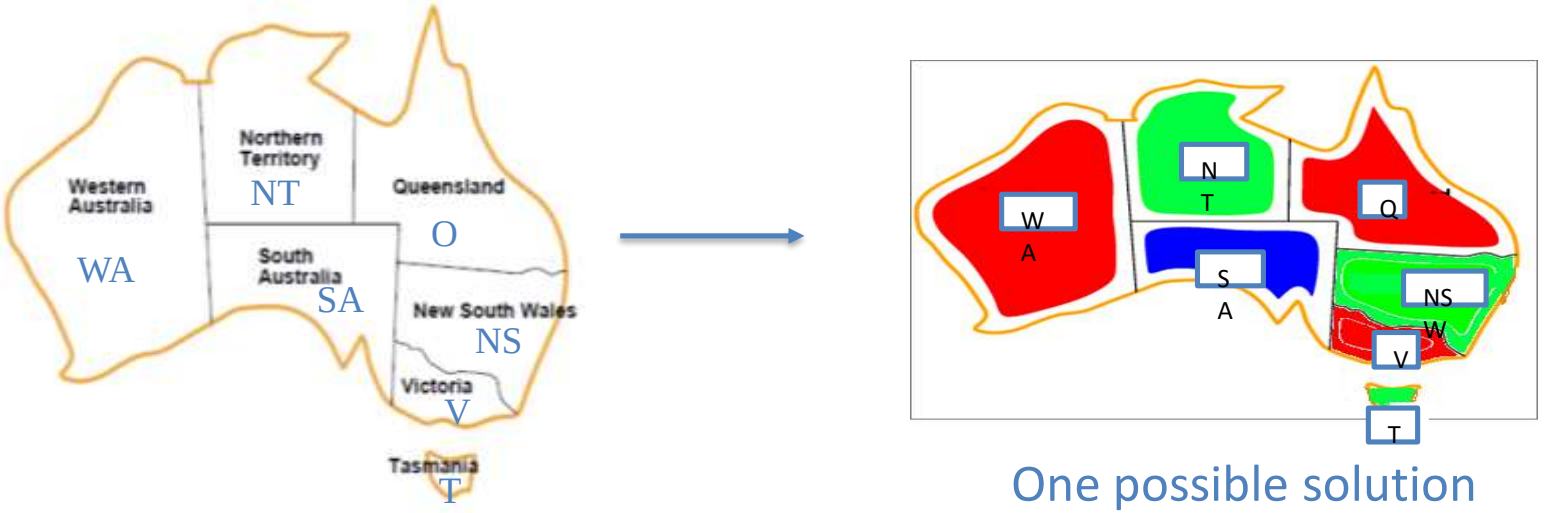
Variables: *WA, NT, Q, NSW, V, SA, T*

Domain: D_i=*{red, green, blue}*

Constraints: adjacent regions must have different colors

e.g., *WA <>NT* (if the language allows this), or

(WA,NT) $\in \{(red, green), (red, blue), (green, red), (green, blue), \dots\}$



One possible solution

Introduction

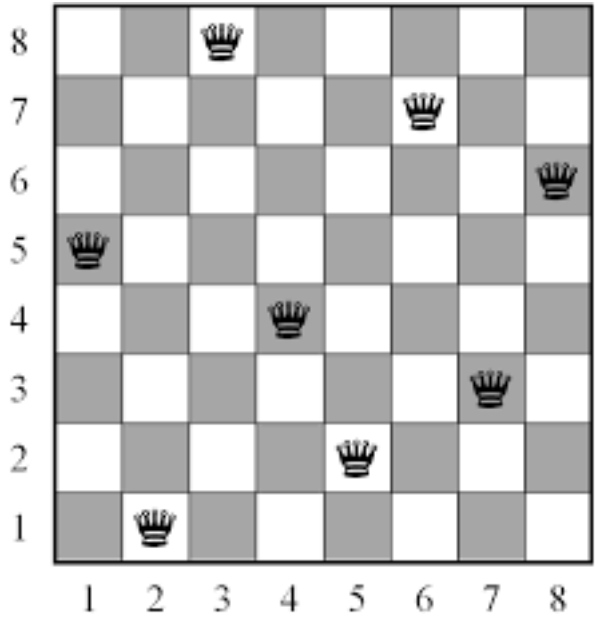
8-Queens problem example:

Variables: The eight queens' positions.

Domain: The Chessboard squares.

Constraints: No queen attacks the other (row, column, and diagonal).

Goal: *To find an "Assignment" of variables satisfying all the constraints.*



Constraint Satisfaction Problem (CSP)

- **Main advantages:**
  - Simple example of a formal representation language
  - Allows useful general-purpose algorithms with more power than standard search algorithms
    - By eliminates large portions of the search space (i.e., by identifying variable/value combinations that violate the constraints).

A CSP is defined by

- $X = \{X_1, X_2, \ldots, X_n\}$ set of n objects/variables.
- $D = \{D_1, D_2, \ldots, D_n\}$ set of finite domains.
- $C = \{C_1, C_2, \ldots, C_m\}$ *set* of constraints, ➔ C_i is a constraint on a subset of objects/variables
- $R = (R_1, R_2, \ldots, R_m)$ set of relationships.
- A **solution** is a complete assignment satisfying all constraints.
- **Goal:** existence of a solution?

Constraint Satisfaction Problem (CSP)

Example of a CSP:

$X=\{X_1, X_2, X_3, X_4\}$ *(4 variables)*

$D_1=D_2=D_3=D_4=\{a, b\}$ *(D_i is a domain with 2 possible values for each variable)*

$C=\{C_{123}, C_{14}, C_{34}\}$ *(C_{14} means X_1 and X_4 has a relationship in a constraint)*

$R=\{R_{123}, R_{14}, R_{34}\}$

Relationships

$R_{123}=\{(a,a,a),(a,b,a),(b,a,b)\}$

$R_{14}=\{(a,a),(b,b)\}$

$R_{34}=\{(a,a),(b,a)\}$

Possible solutions:

- *(a,a,a,a)*

$\Rightarrow$ *$X_1 = a, X_2 = a, X_3 = a , X_4=a$*

- *(a,b,a,a)*

$\Rightarrow$ *$X_1 = a, X_2 = b, X_3 = a , X_4=a$*

*R_{14}: either (a,a): $X_1=a$ **and** $X_4 = a$*

OR *(b,b): $X_1=b$ **and** $X_4 = b$;*

i.e., (b,a): $X_1=b$ and $X_4 = a$ violates R_{14}

Variables of CSPs

- The simplest kind of CSP involves variables that have **discrete**, **finite** domains.

- **Discrete variables**
  - **finite domains:**
    - *n* variables, domain size *d* → $O(d^n)$ complete assignments
    - e.g., Boolean CSPs, incl. Boolean satisfiability (NP-complete), SAT
    - e.g., Map-coloring problems, scheduling with time limits, the 8-queens problem
  - **infinite domains:**
    - integers, strings, etc.
    - e.g., job scheduling with no deadline, variables are start/end days for each job, infinite number of start times for each variable.
    - need a constraint language, *e.g., StartJob1 + 5 ≤ StartJob3*

Variables of CSPs

- The simplest kind of CSP involves variables that have **discrete**, **finite** domains.

- **Continuous variables**
  - e.g., start/end times for Hubble Space Telescope observations
  - linear constraints solvable in polynomial time by linear programming

Constraint types

- **Types of constraints**
  - Temporal (e.g., $t_i > t_k + d$)
  - Geometric (spatial) (e.g., *distance (x, y) <= d*)
  - Algebraic, numeric (e.g., $x >= y + z$)
  - Logic ... (e.g., $x \wedge y \Rightarrow z$)
  - Symbolic : any no-numerical constraint.
- **Constraint in intention:**
  - $t_i > t_j + d$
  - $x \wedge y \Rightarrow z$
- **Constraint in extension:**
  - C_{xyz} constraint on *X, Y, Z.*
  - R_{xyz}={(a,a,a),(a,b,a), (b,a,b)}

Previous example

C={C_{123}, C_{14}, C_{34}} (C_{14} means X_1 and X_4 has a relationship in a constraint)

R_{123}={(a,a,a),(a,b,a), (b,a,b)}

R_{14}={(a,a),(b,b)}

R_{34}={(a,a), (b,a)}

Varieties of constraints

- **Unary** constraints involve a single variable,
  - e.g., *SA* ≠ *green color*
- **Binary** constraints involve pairs of variables,
  - e.g., *SA* ≠ *WA*
- **Higher-order** constraints involve 3 or more variables,
  - e.g., $x_i > x_j > x_k$ OR **cryptarithmetic column constraint**

Varieties of constraints - Cryptarithmetic puzzle

- A type of mathematical game consisting of a mathematical equation among unknown numbers, whose digits are represented by letters of the alphabet.
- The goal is to identify the value of each letter.
- The name can be extended to puzzles that use non-alphabetic symbols instead of letters.
- The equation is typically a basic operation of arithmetic, such as addition, multiplication, or division.

Varieties of constraints - Cryptarithmetic puzzle

- A classic example (1924):

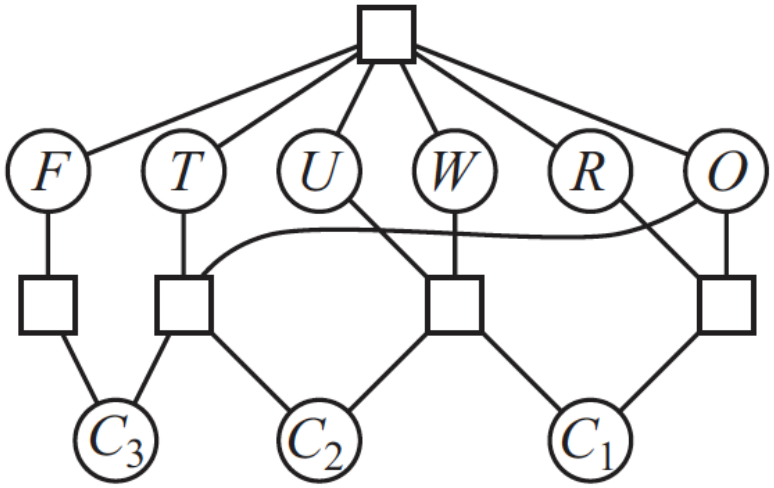
$$\begin{array}{cccccc} & & S & E & N & D \\ + & & M & O & R & E \\ \hline = & M & O & N & E & Y \end{array}$$

- The solution to this puzzle is O = 0, M = 1, Y = 2, E = 5, N = 6, D = 7, R = 8, and S = 9.

- Traditionally, each letter should represent a **different** digit, and (as an ordinary arithmetic notation) the **leading digit** of a multi-digit number **must not be zero**.
- A good puzzle should have a unique solution, and the letters should make up a phrase.

Varieties of constraints - Cryptarithmetic puzzle

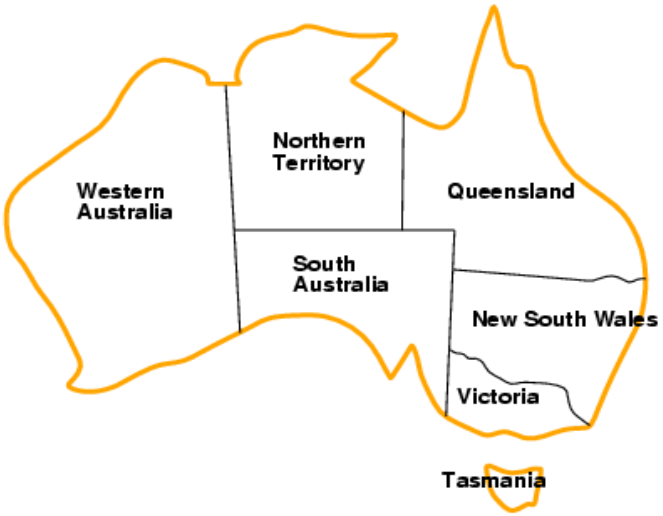
$$\begin{array}{r} T\ W\ O \\ +\ T\ W\ O \\ \hline F\ O\ U\ R \end{array}$$

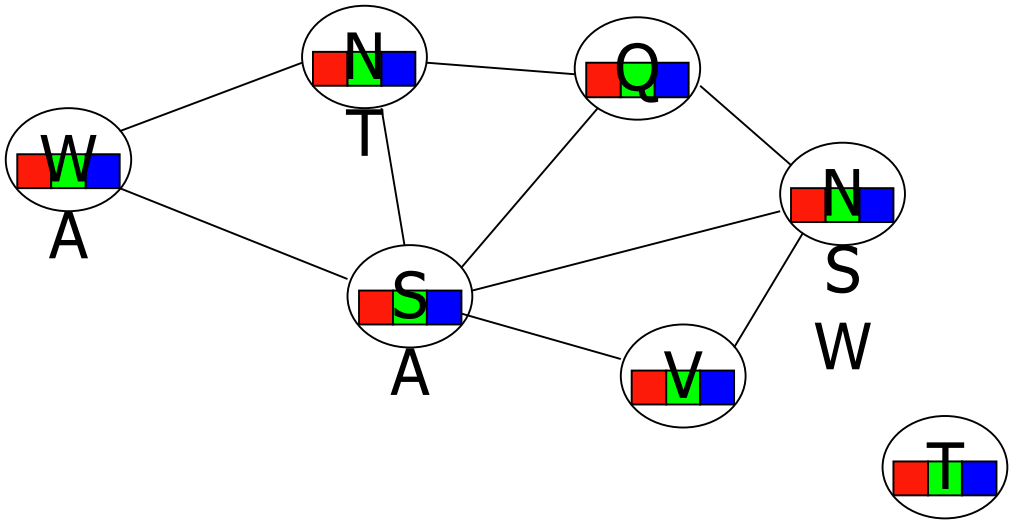


$$Alldiff(F, T, U, W, R, O)$$
$$O + O = R + 10 \cdot C_{10}$$
$$C_{10} + W + W = U + 10 \cdot C_{100}$$
$$C_{100} + T + T = O + 10 \cdot C_{1000}$$
$$C_{1000} = F\ ,$$

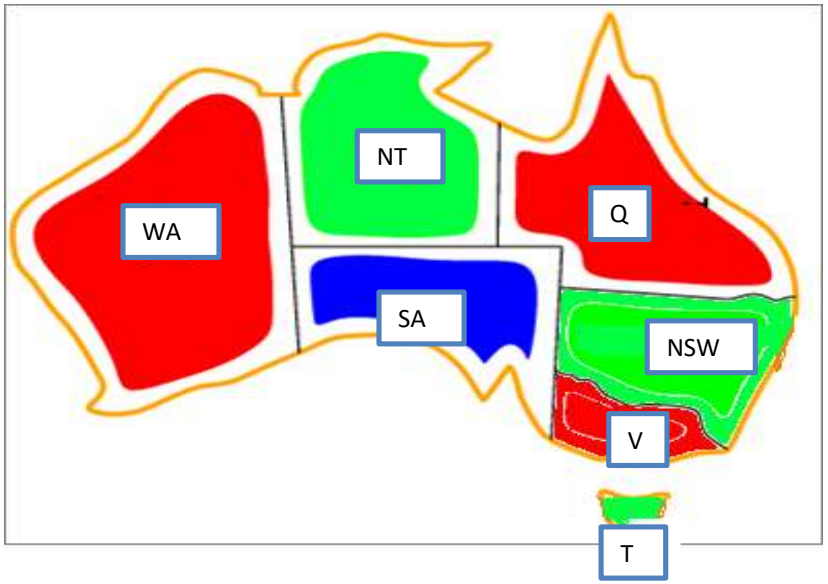
Varieties of constraints - Constraint Graph

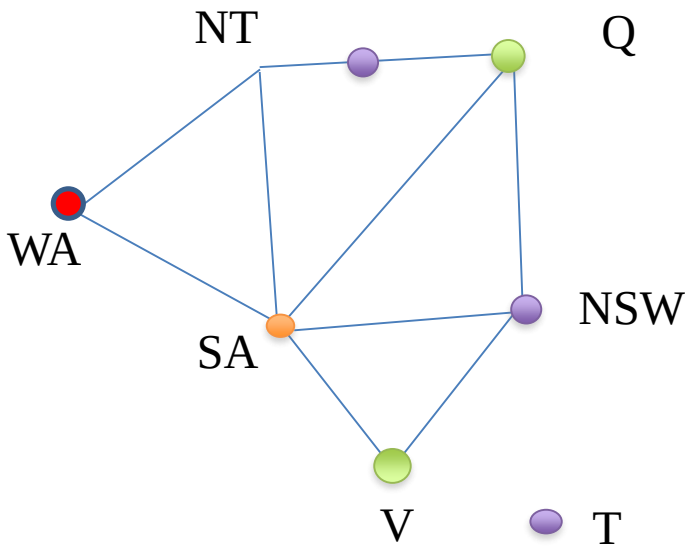
- It can be helpful to visualize a CSP as a **constraint graph.**
  - Nodes of the graph correspond to variables of the problem, and a link connects any two variables that participate in a constraint.





Example: Map-Coloring



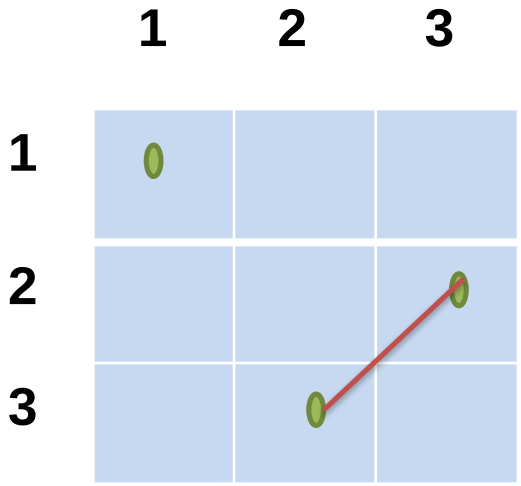


Solution :

WA=red, NT=green, Q=red, NT=green, V=red, SA=blue, T=green

Binary CSP: N-queens problem

- N-queens problem : place n queens on nxn-chessboard so that no queen attacks another.
- **N = 3**



CSP formulation

$Q=\{Q_1, Q_2, Q_3\}$ variables

$D_1=D_2=D_3=\{c_1,c_2,c_3\}$ domains\values

Relationship:

$R_{12}=\{(c_1,c_3),(c_3,c_1)\}$

$R_{13}=\{(c_1,c_2),(c_2,c_1),(c_2,c_3),(c_3,c_2)\}$

$R_{23}=\{(c_1,c_3),(c_3,c_1)\}$

No solution.

Representing the Constraints :

1. Between every pair of variables (Q_i,Q_j) $(i \neq j)$, we have a constraint c_{ij}.

2. For each c_{ij}, an assignment of values to the variables $Q_i = A$ and $Q_j = B$, satisfies this constraint if and only if

1. $A \neq B$

2. $|A-B| \neq |i-j|$

Standard Search Formulation (Incremental)

- ***n*** variables
- domain size ***d***
- Branching factor: ***(n-1) * d***
  - The branching factor at the top level is ***nd*** because any of d values can be assigned to any of ***n*** variables. At the next level, the branching factor is ***(n – 1)d***, and so on for ***n*** levels.
- Number of leaves: $n! * d^n$
- But there are only d^n complete assignments!

– What went wrong?

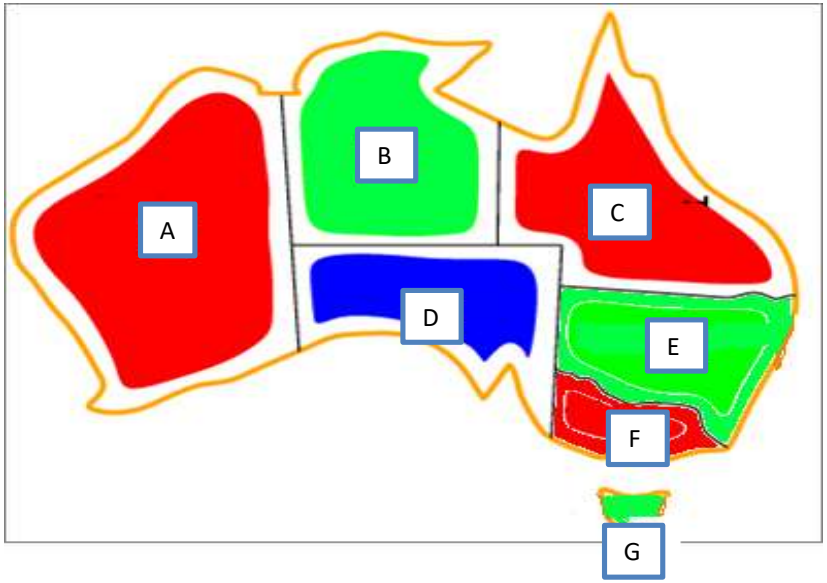
CSP Search algorithms:

- Backtracking (BT)
- Forward Checking

Standard Search Formulation (Incremental)

- In regular state-space search, an algorithm can only search.
- In CSPs:
  - an algorithm can search (choose a new variable assignment from several possibilities) or
    - Backtracking search
  - do a specific type of inference called constraint propagation.
    - AC-3 (before search)
    - Forward Checking (during the search)

Example: Map-Coloring



Map-coloring problem

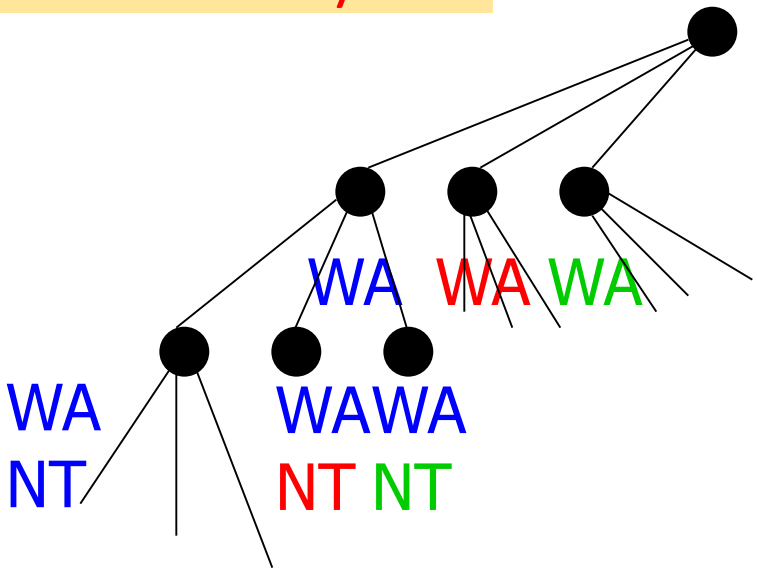
- Modeled as a CSP
- Solve it with BT algorithm

Uninformed Backtracking (Depth-First) search

- Special property of CSPs: they are commutative:
  - This means: the order in which we assign variables does not matter.

NT WA = WA NT

- Need only consider a single variable at each node in the search tree.
- Better search tree: (1) order variables (2) assign them values one-by-one.



D

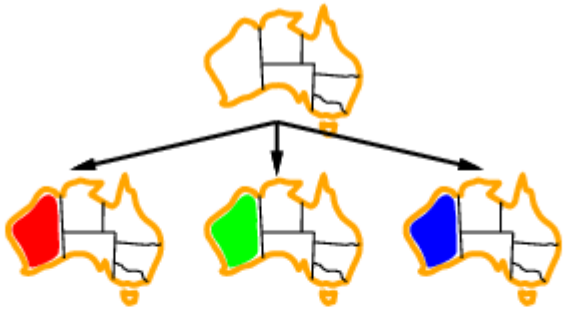
D^2

D^N

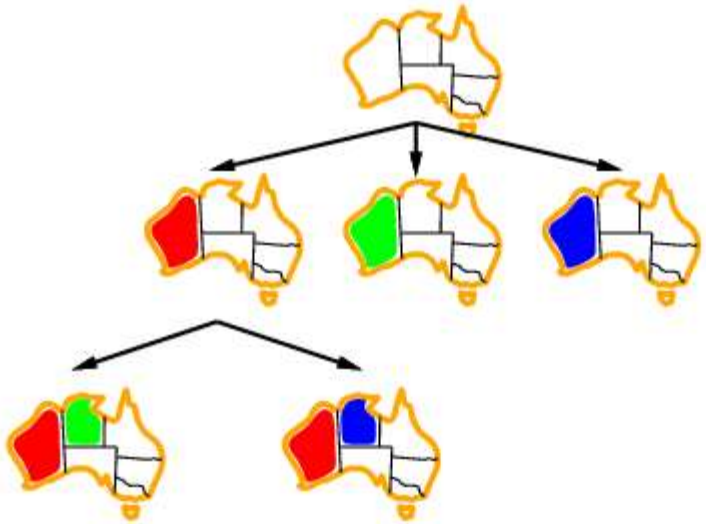
BT algorithm : example



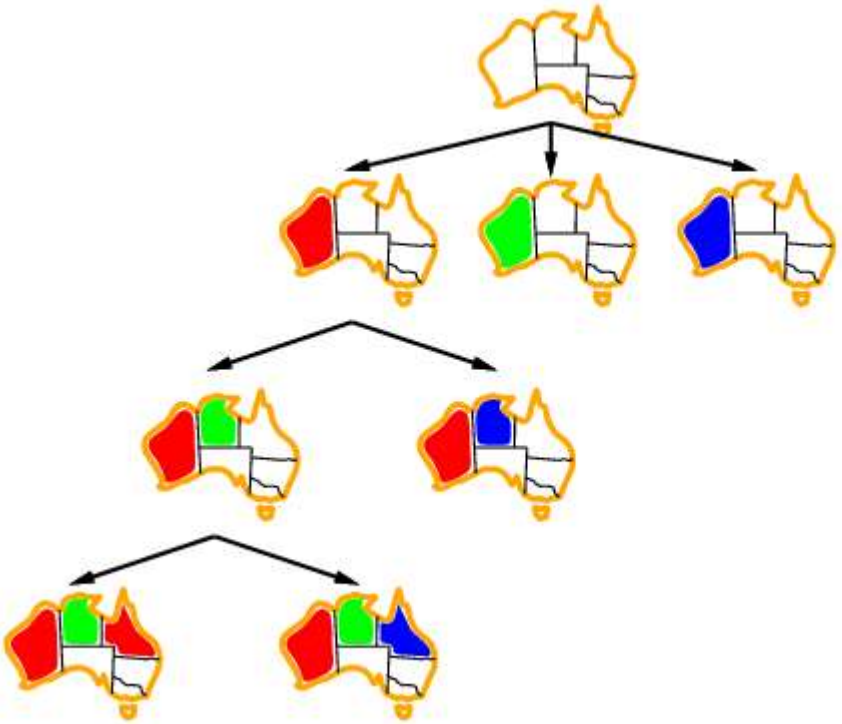
BT algorithm : example



BT algorithm : example



BT algorithm : example



Time complexity $O(d^n)$:

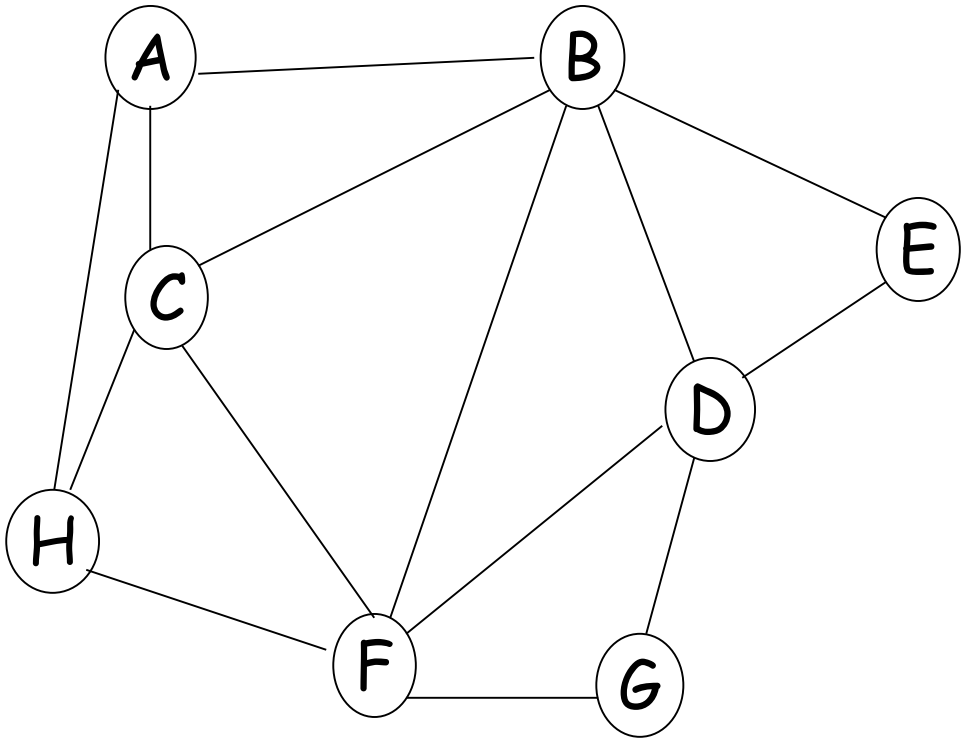
Case n=50 and d=2, time complexity : 2^{50}

Each case takes 1 ms, 2^{50} ms =37000 years

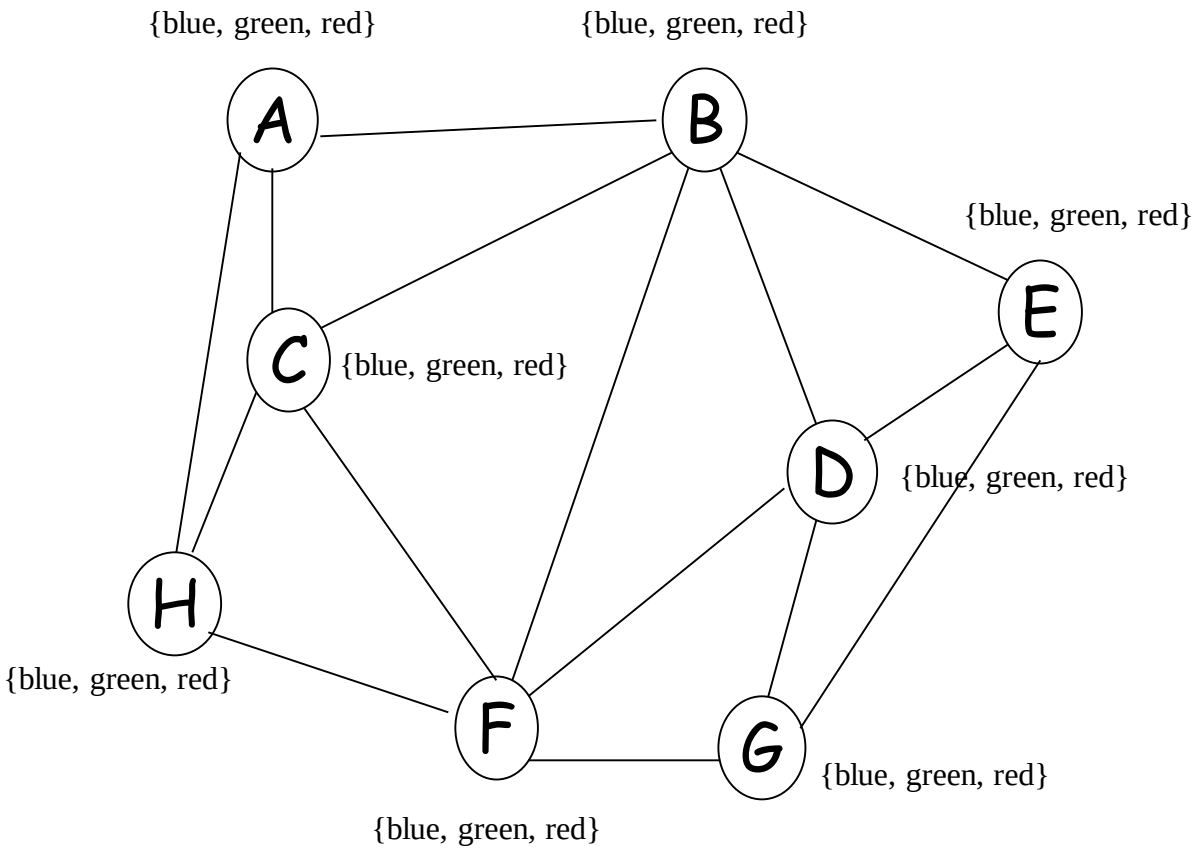
Uninformed Backtracking search (BT)

- Backtracking (BT) search is a depth-first search that chooses values for one variable at a time and backtracks when a variable has no legal values left to assign.
- Every solution appears at depth *n* with *n* variables → use depth-first search.
- Backtracking search is the basic uninformed algorithm for CSPs.
- Because the representation of CSPs is standardized, there is no need to supply BT search with a domain-specific initial state, action function, transition model, or goal test
- Can solve *n*-queens for $n \approx 25$

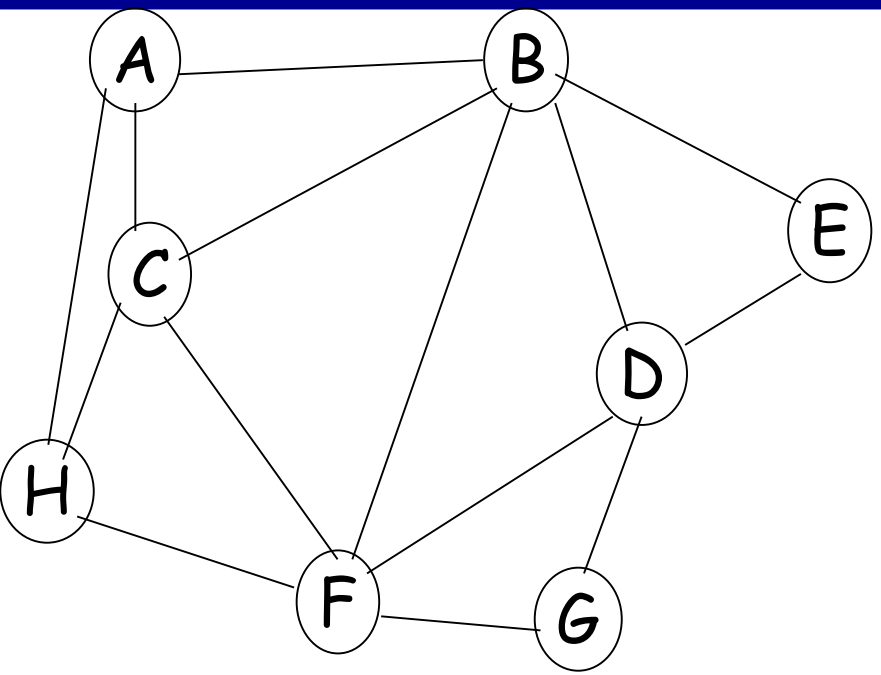
Uninformed BT Algorithm: Map-Coloring Example



Uninformed BT Algorithm: Map-Coloring Example

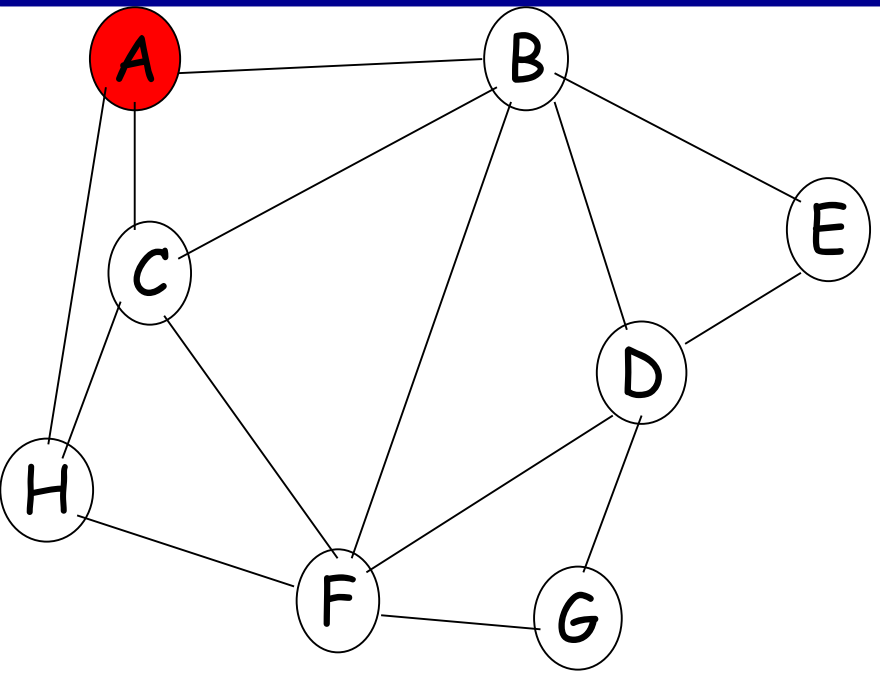


Uninformed BT Algorithm: Map-Coloring Example



1 = red
2 = blue
3 = green

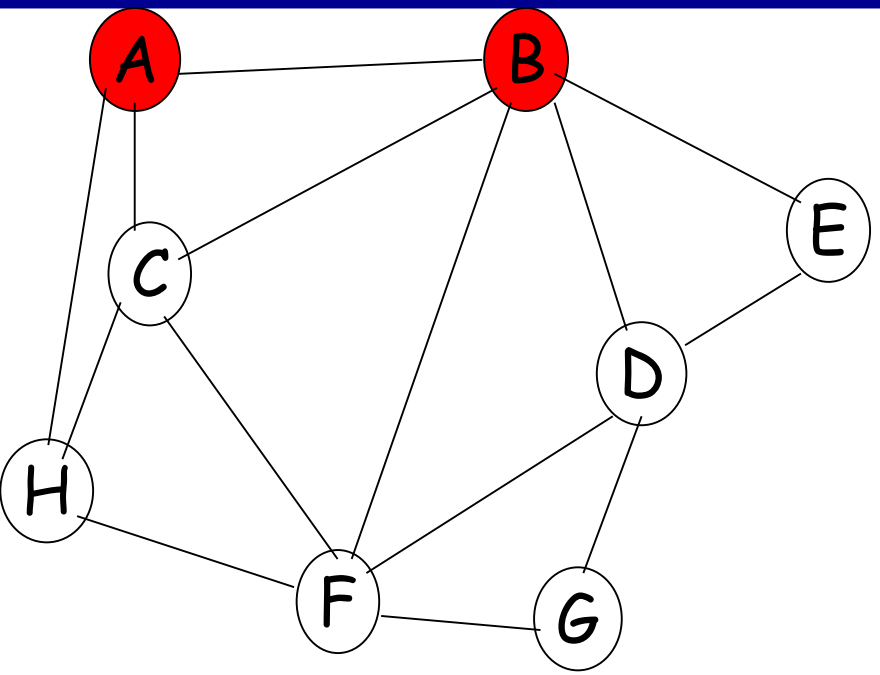
Uninformed BT Algorithm: Map-Coloring Example

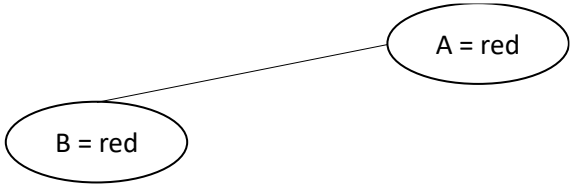


A = red

1 = red
2 = blue
3 = green

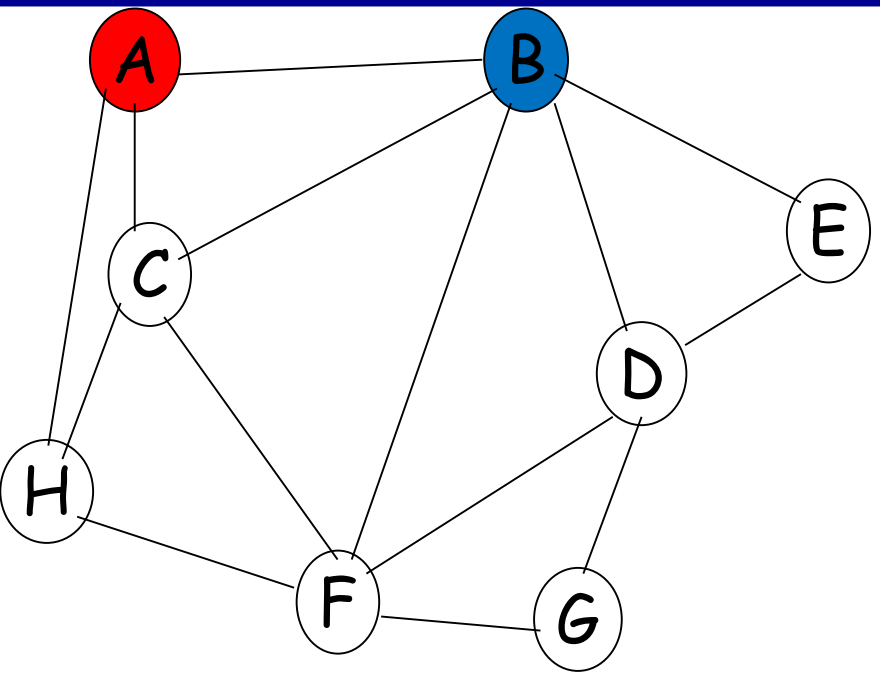
Uninformed BT Algorithm: Map-Coloring Example

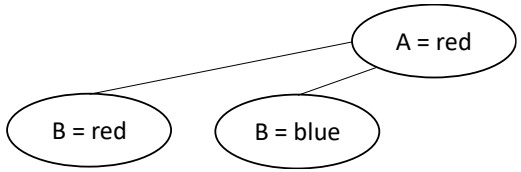




1 = red
2 = blue
3 = green

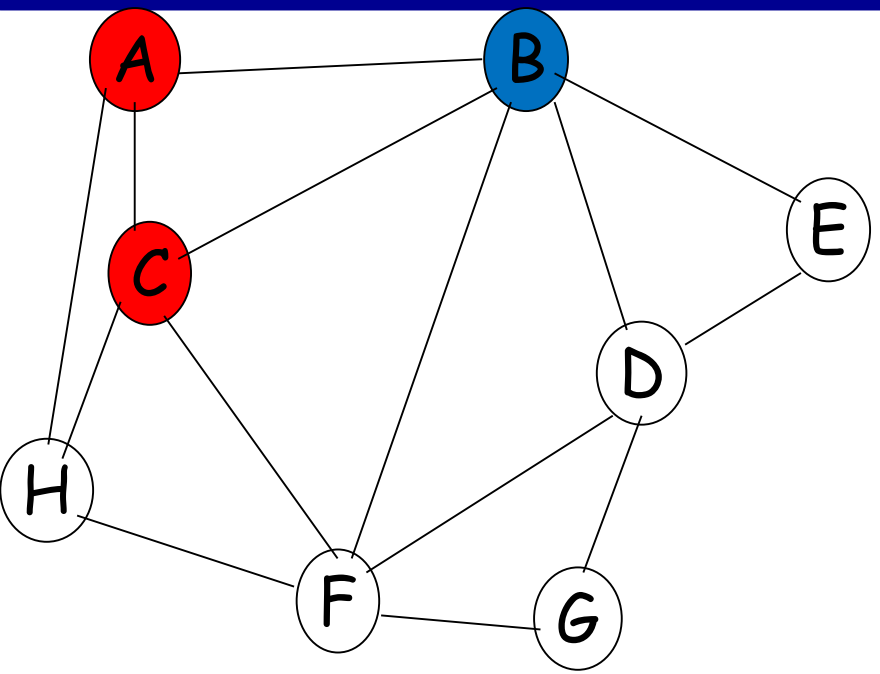
Uninformed BT Algorithm: Map-Coloring Example

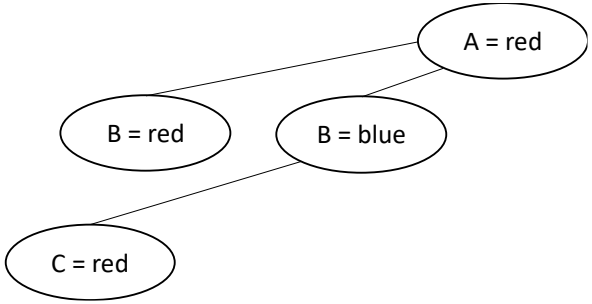




1 = red
2 = blue
3 = green

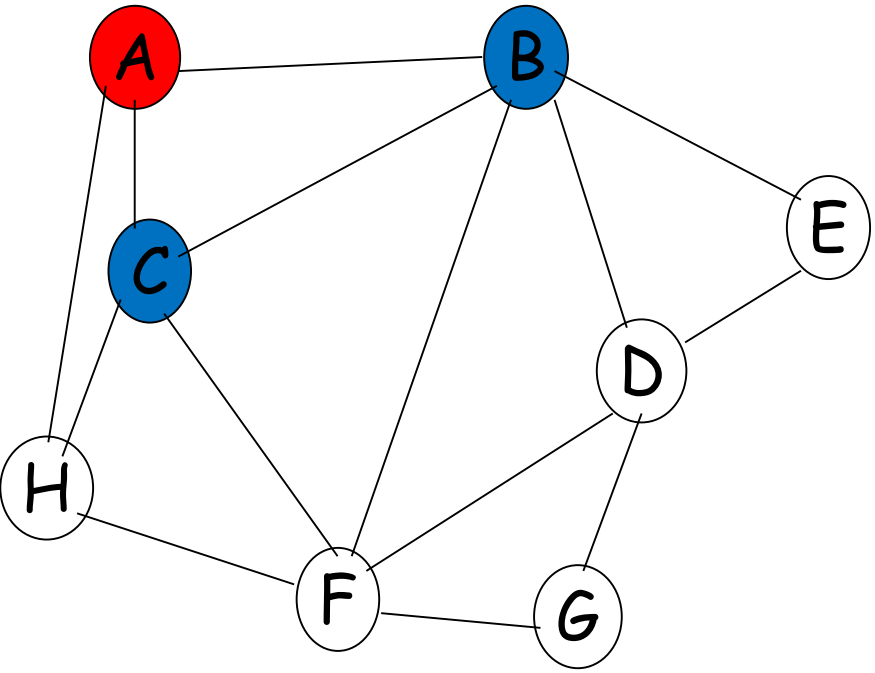
Uninformed BT Algorithm: Map-Coloring Example

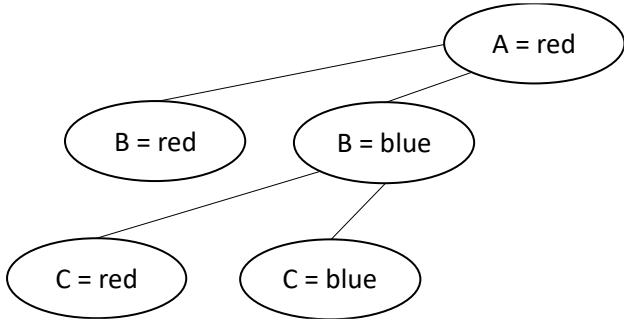




1 = red
2 = blue
3 = green

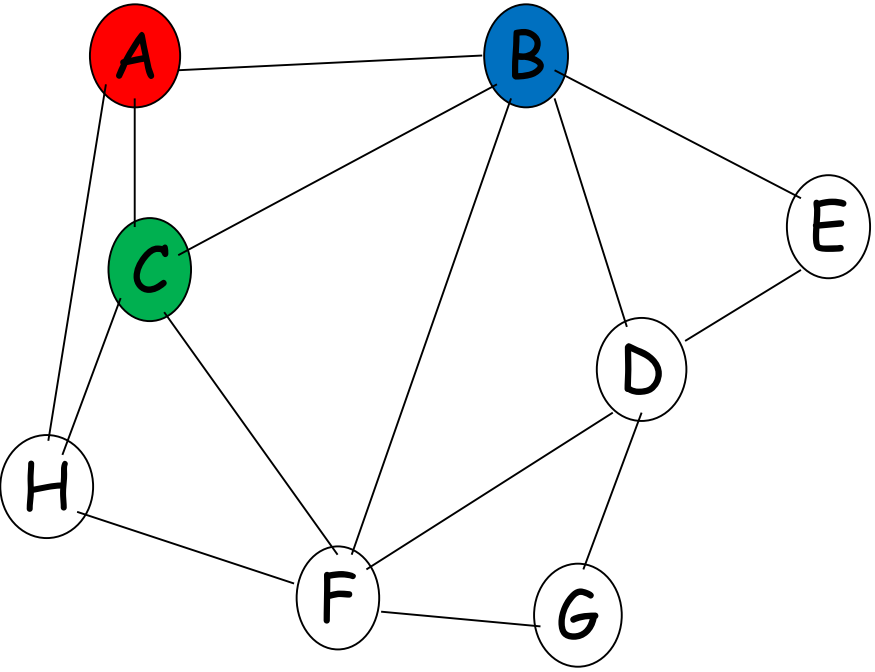
Uninformed BT Algorithm: Map-Coloring Example

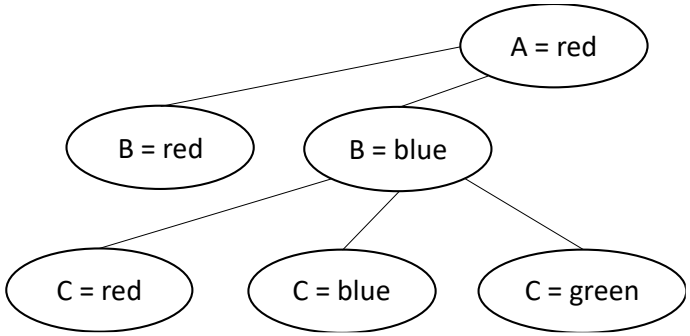




1 = red
2 = blue
3 = green

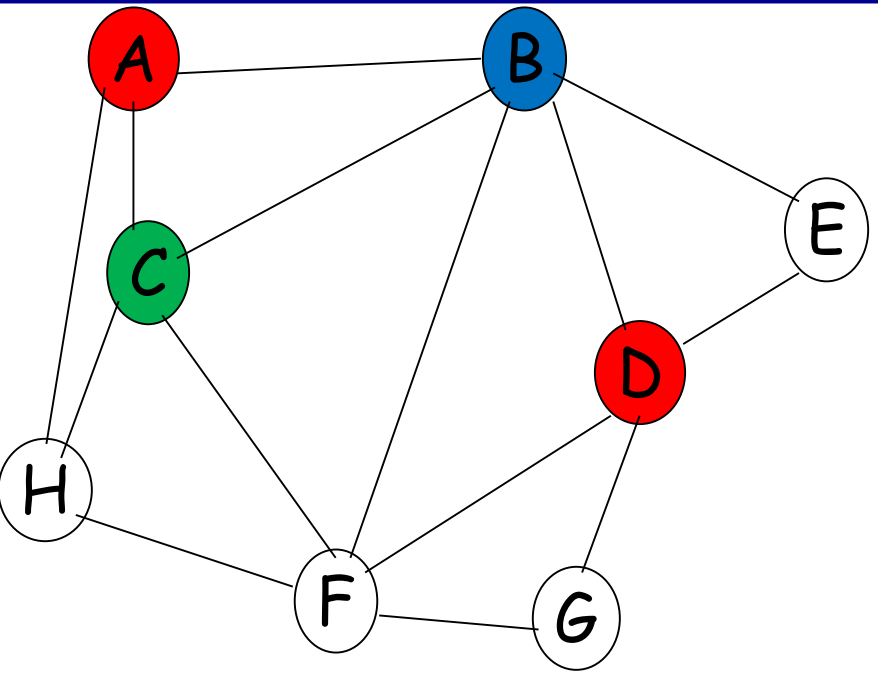
Uninformed BT Algorithm: Map-Coloring Example

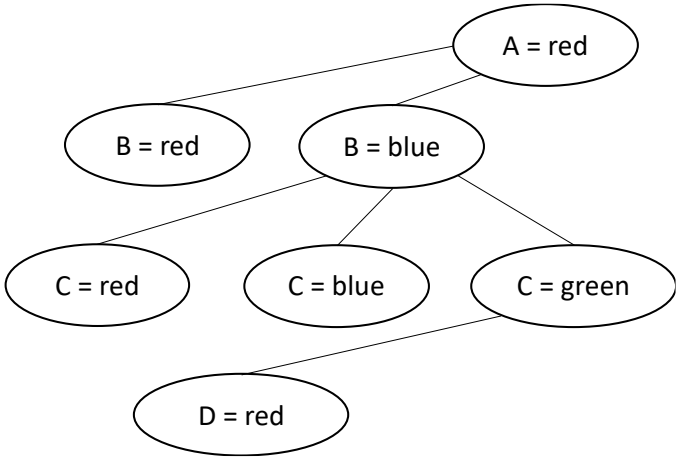




1 = red
2 = blue
3 = green

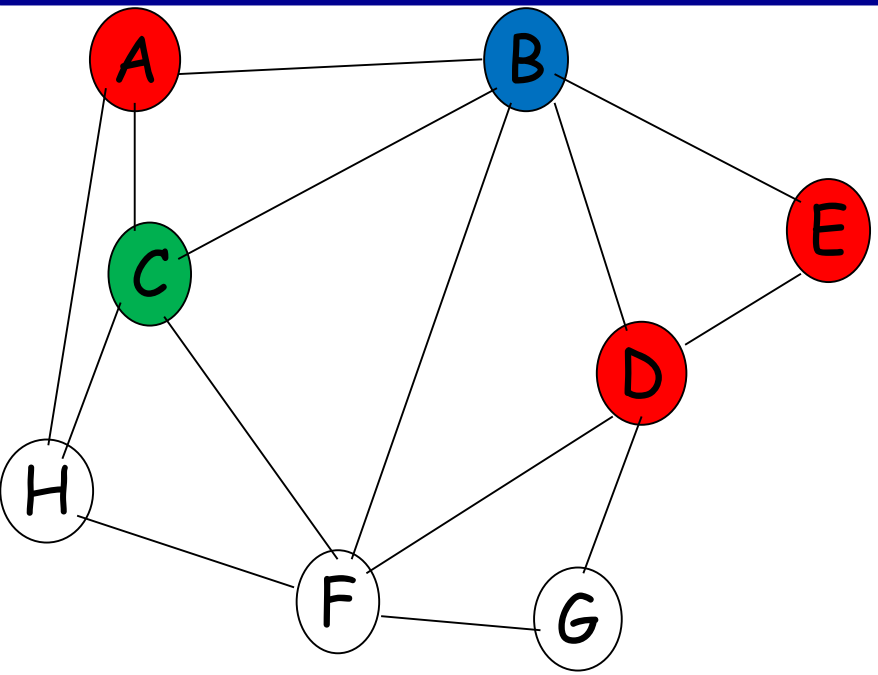
Uninformed BT Algorithm: Map-Coloring Example

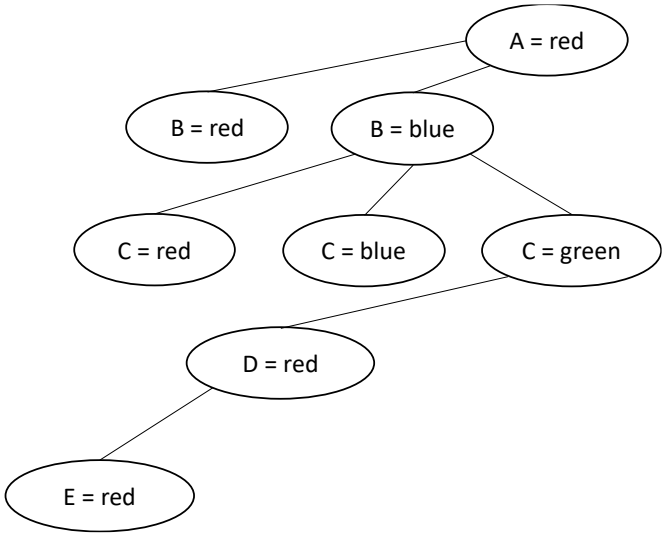




1 = red
2 = blue
3 = green

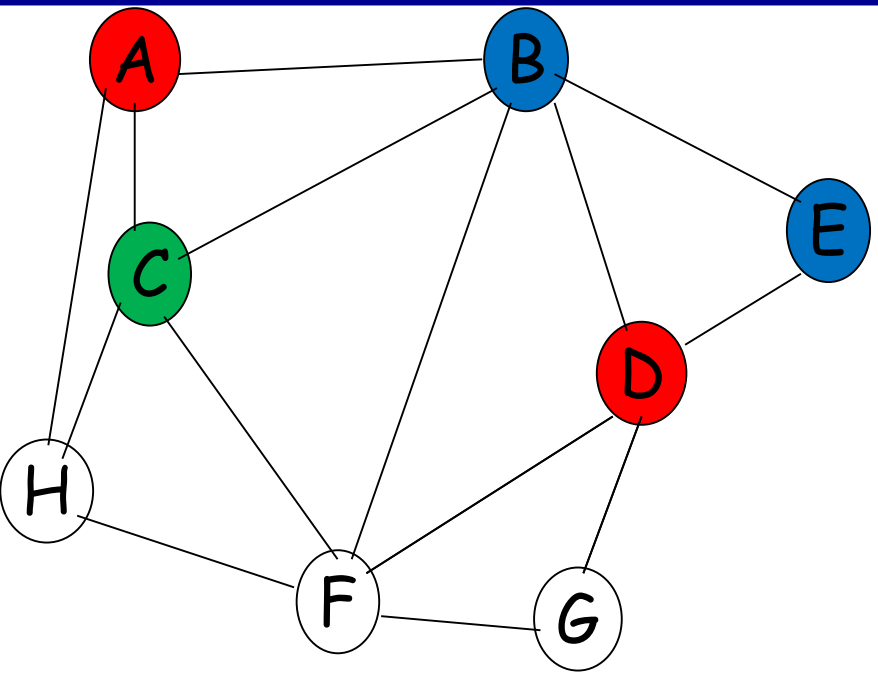
Uninformed BT Algorithm: Map-Coloring Example

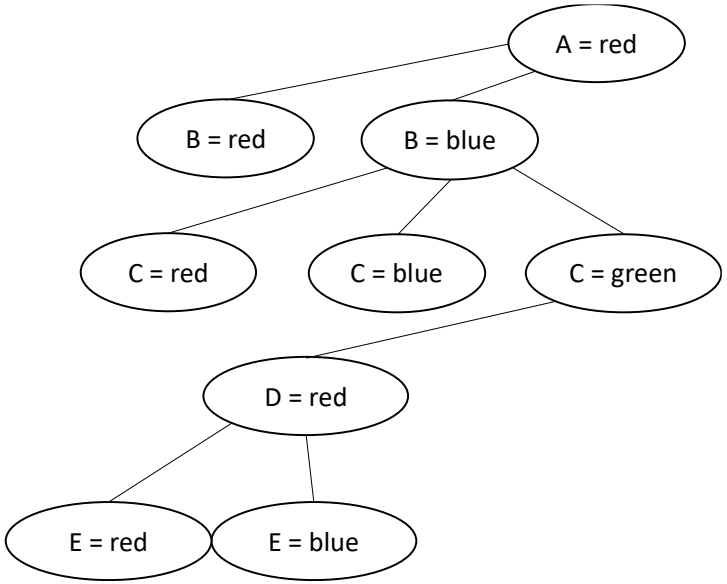




1 = red
2 = blue
3 = green

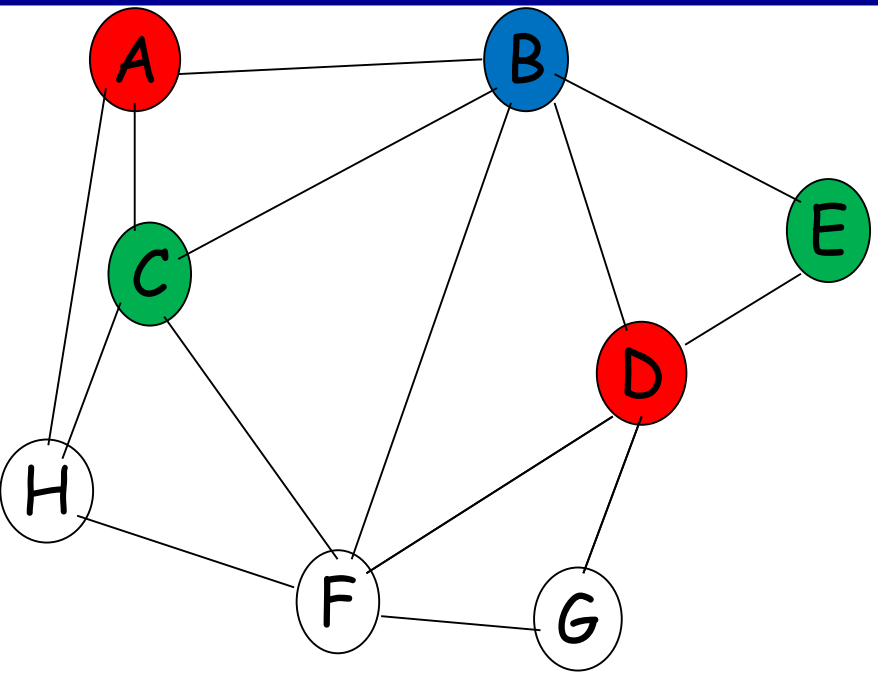
Uninformed BT Algorithm: Map-Coloring Example

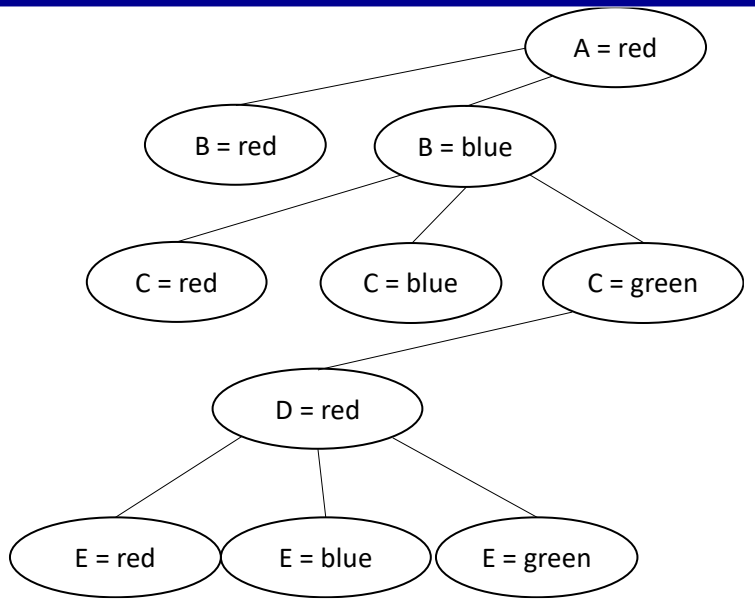




1 = red
2 = blue
3 = green

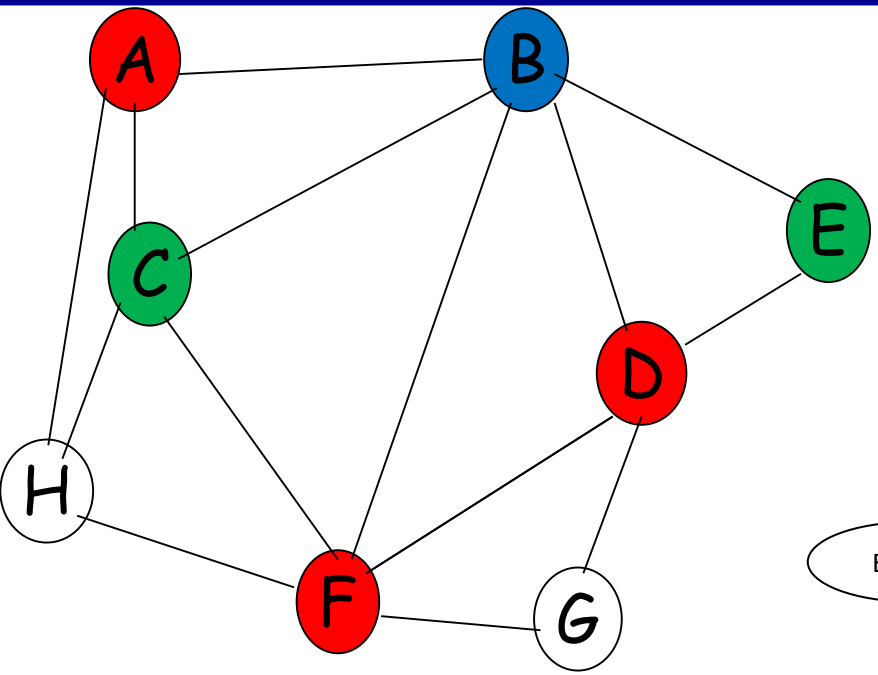
Uninformed BT Algorithm: Map-Coloring Example

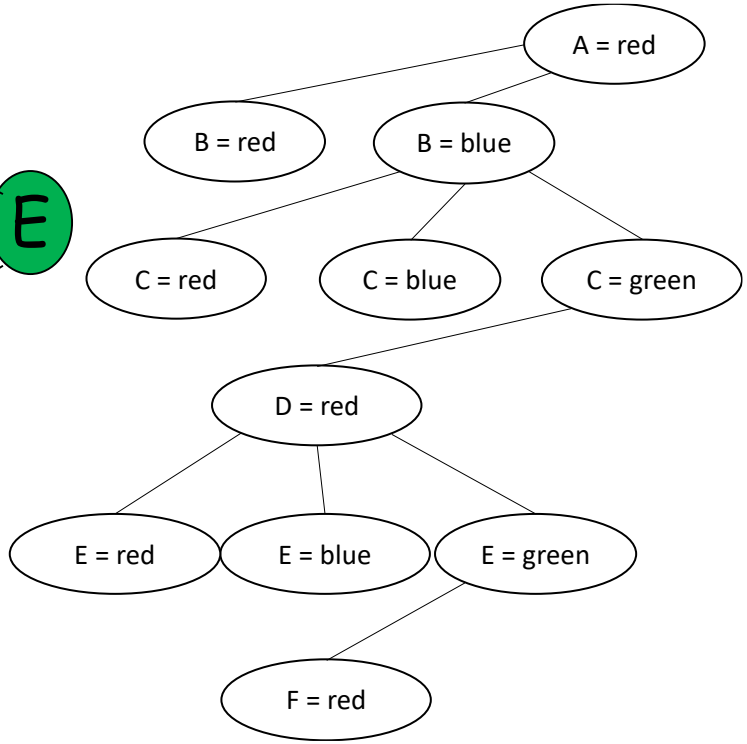




1 = red
2 = blue
3 = green

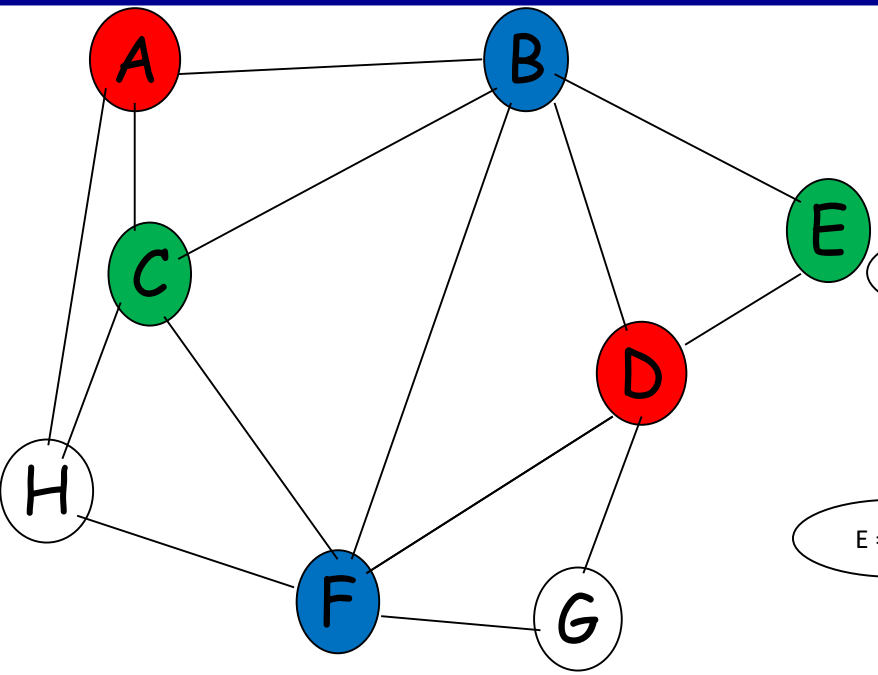
Uninformed BT Algorithm: Map-Coloring Example

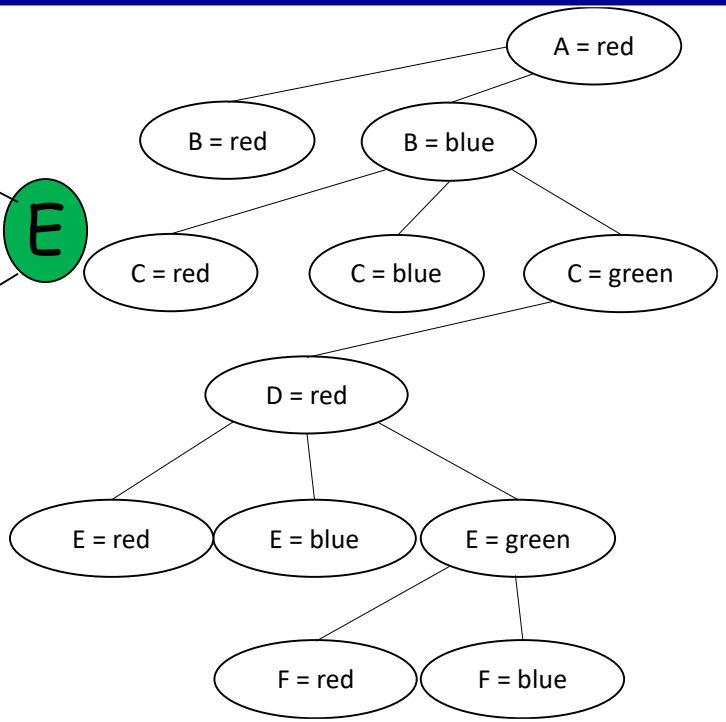




1 = red
2 = blue
3 = green

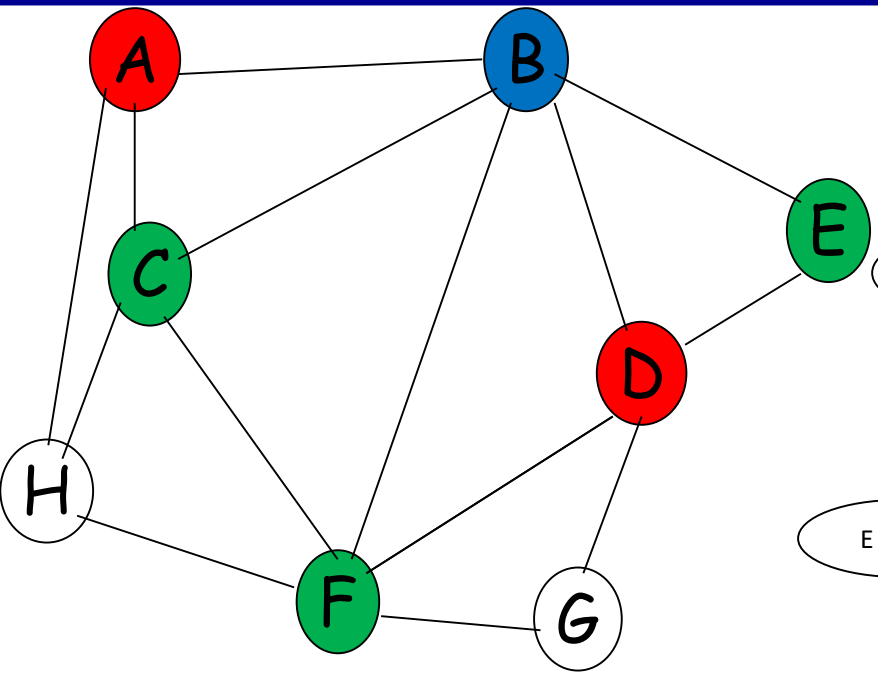
Uninformed BT Algorithm: Map-Coloring Example

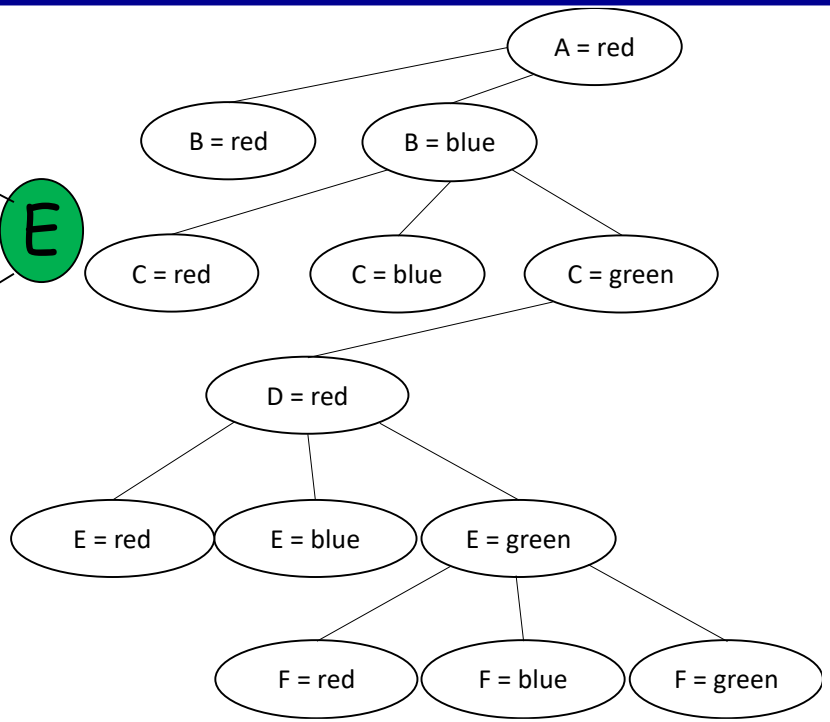




1 = red
2 = blue
3 = green

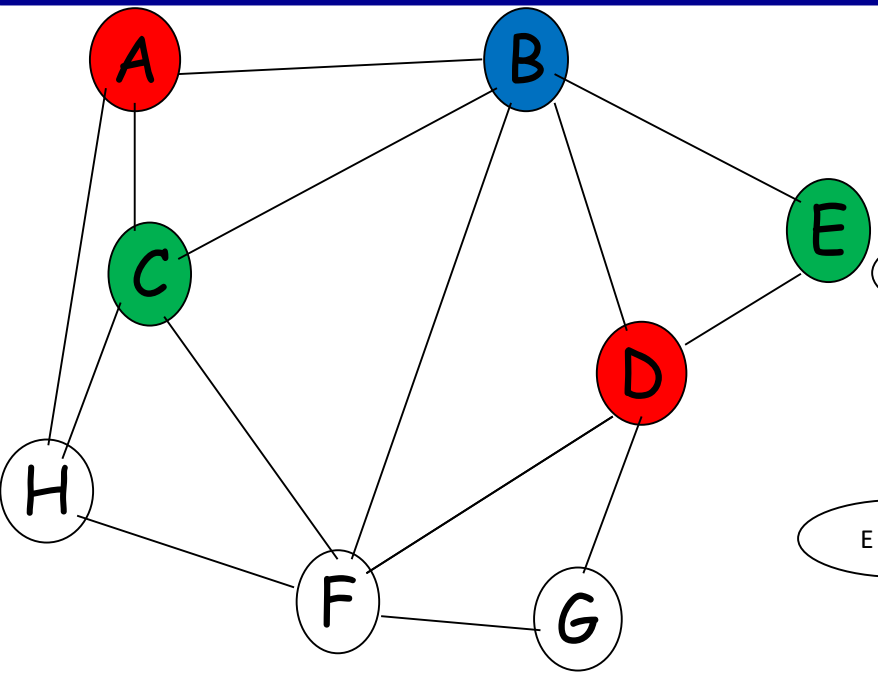
Uninformed BT Algorithm: Map-Coloring Example

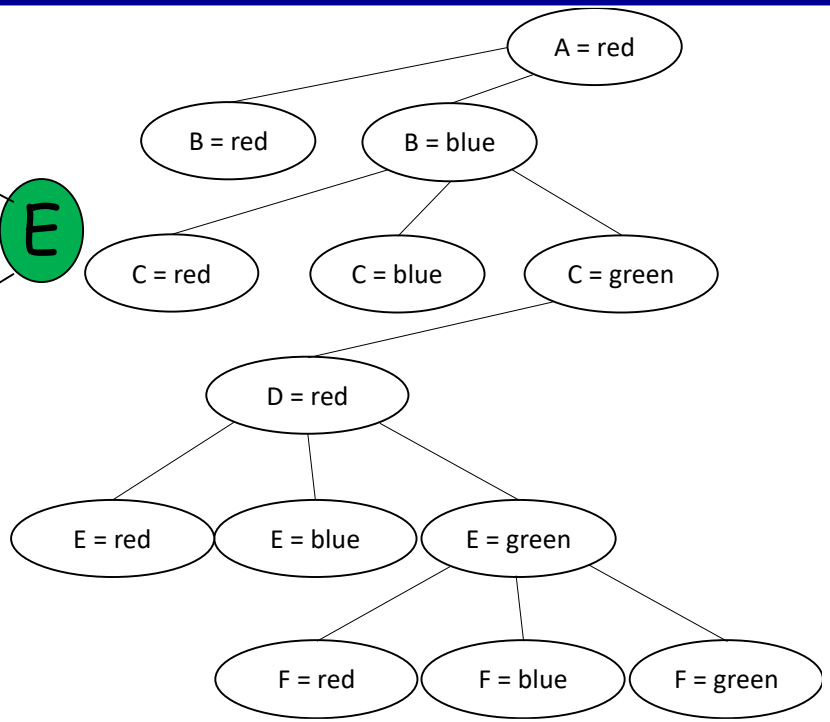




1 = red
2 = blue
3 = green

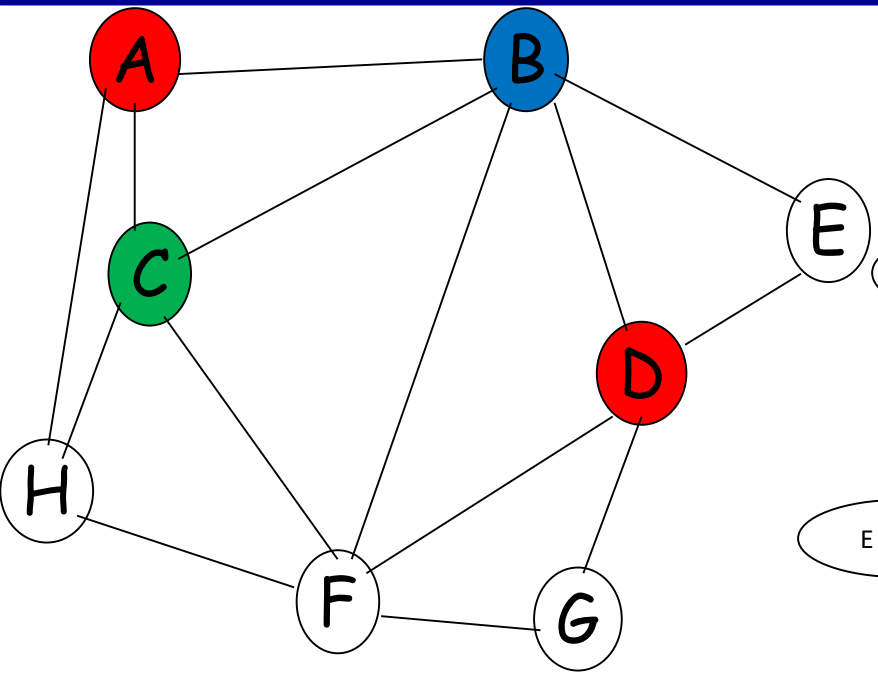
Uninformed BT Algorithm: Map-Coloring Example

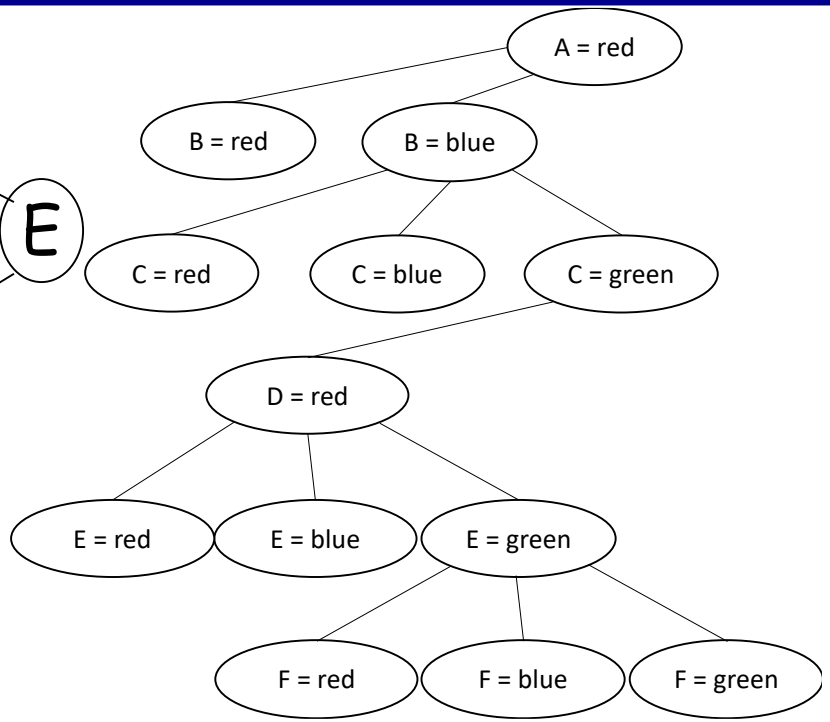




1 = red
2 = blue
3 = green

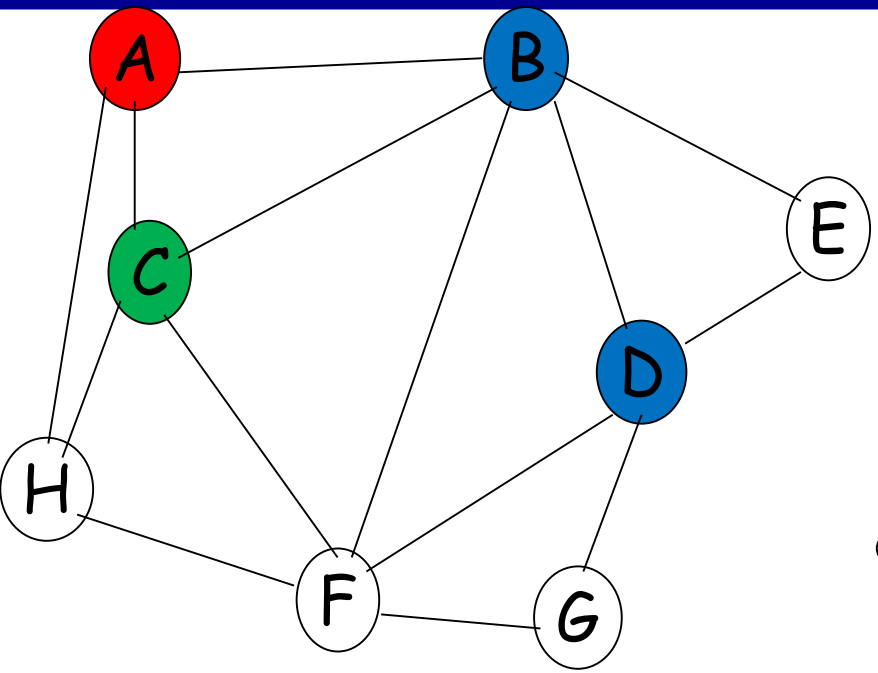
Uninformed BT Algorithm: Map-Coloring Example

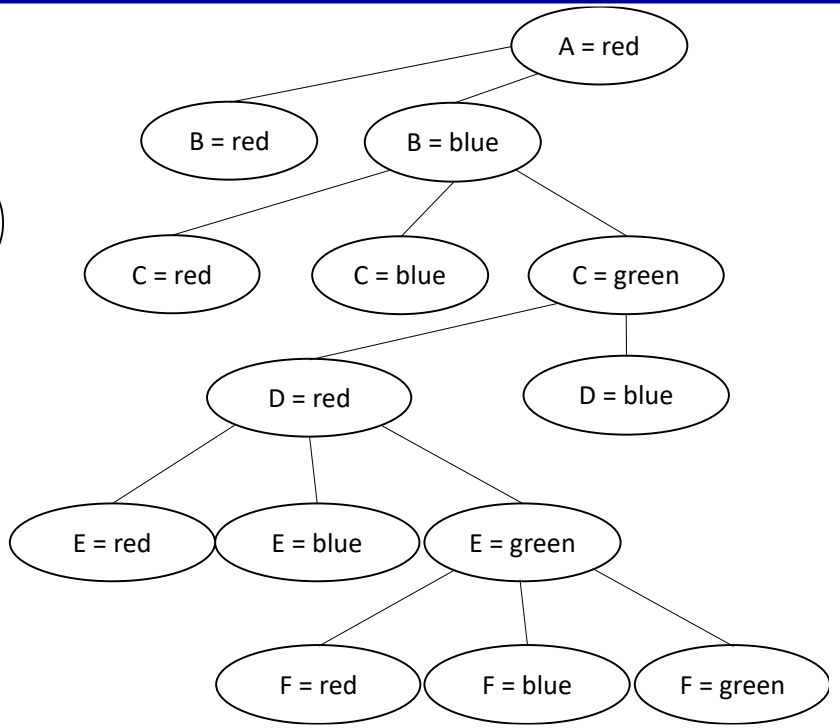




1 = red
2 = blue
3 = green

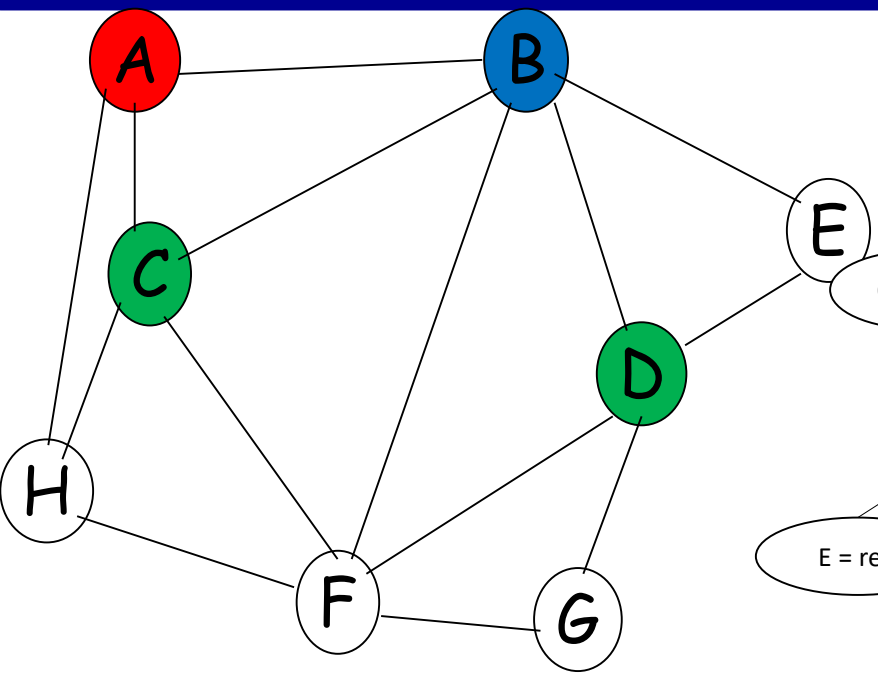
Uninformed BT Algorithm: Map-Coloring Example

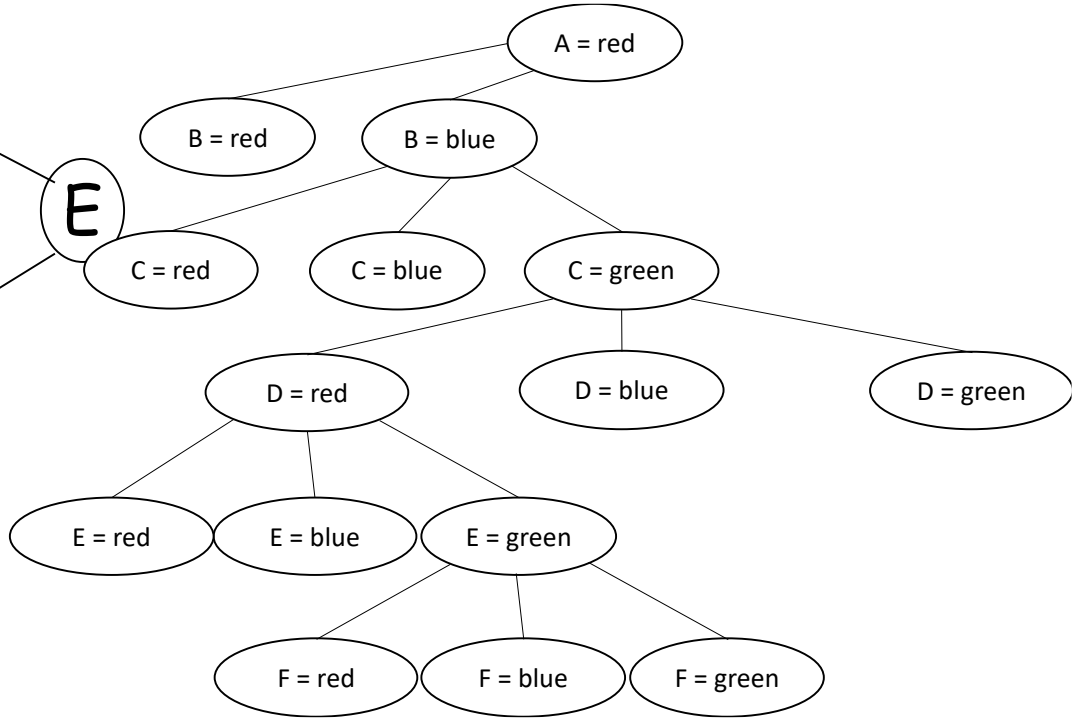




1 = red
2 = blue
3 = green

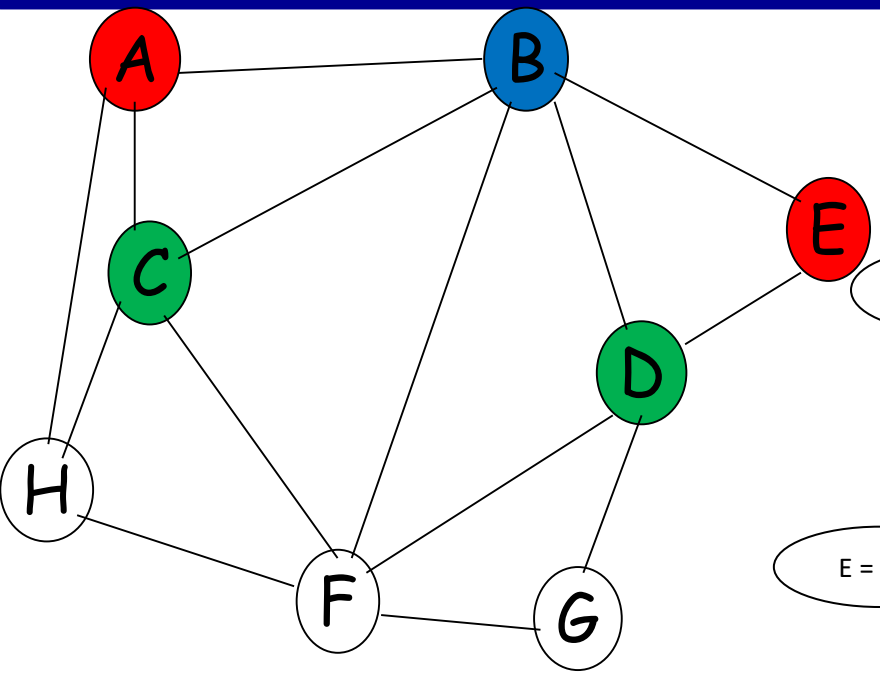
Uninformed BT Algorithm: Map-Coloring Example

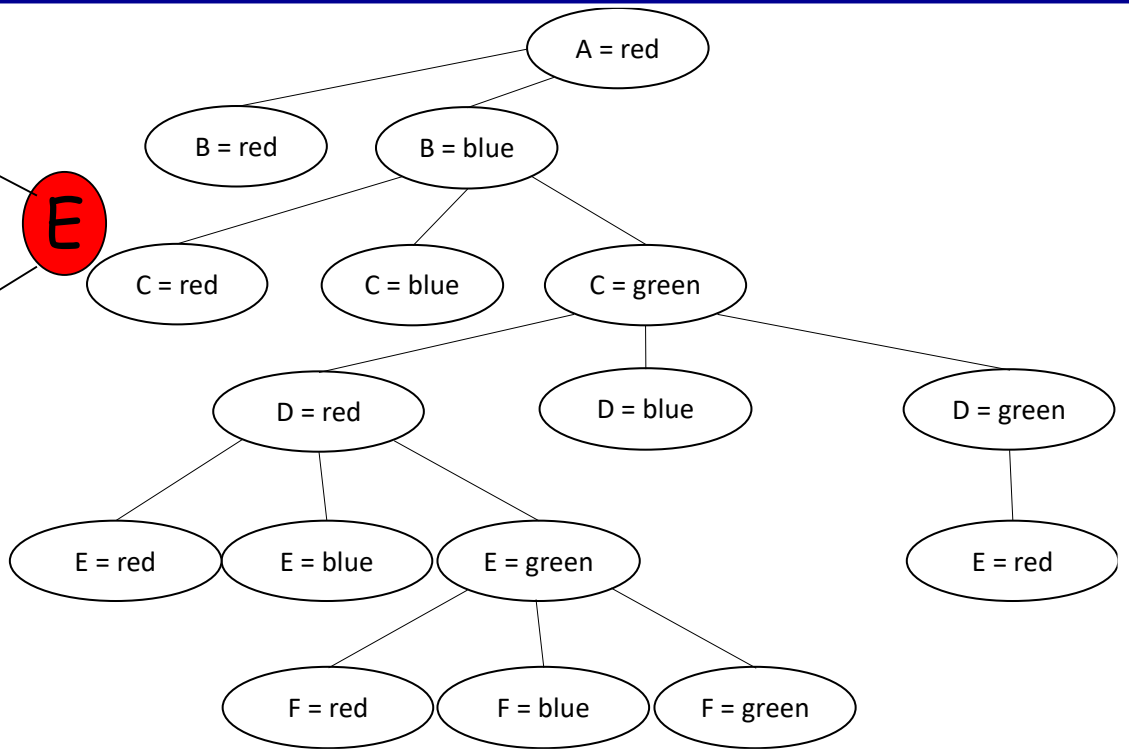




1 = red
2 = blue
3 = green

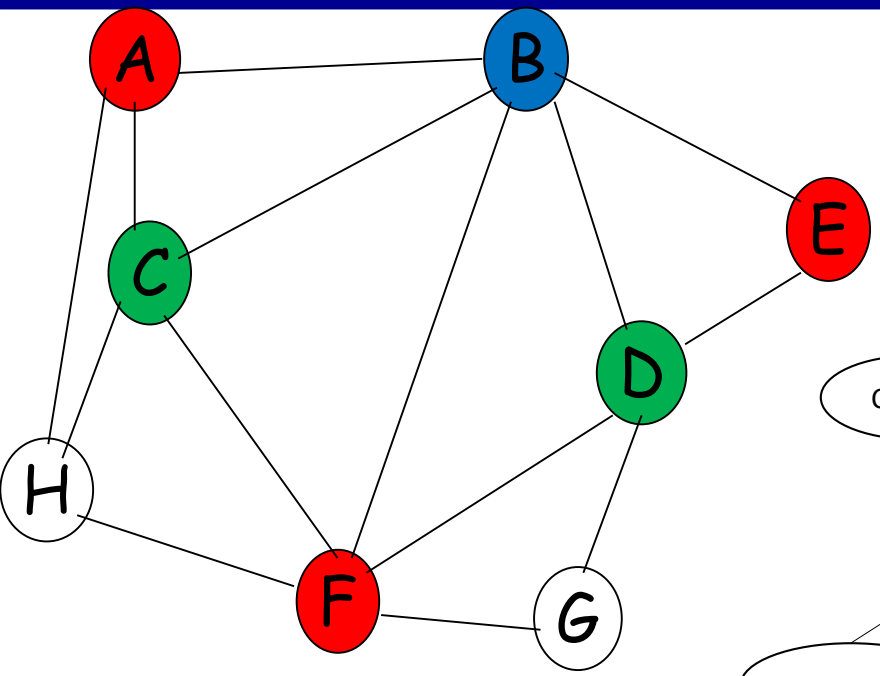
Uninformed BT Algorithm: Map-Coloring Example

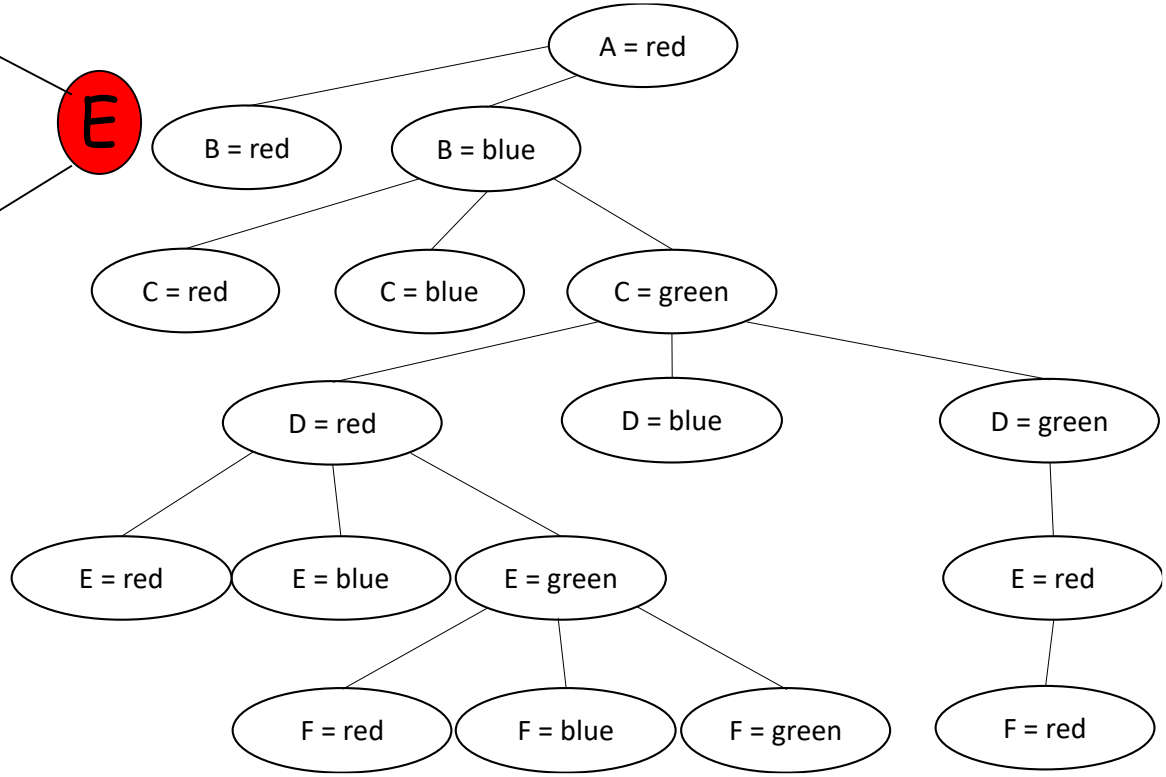




1 = red
2 = blue
3 = green

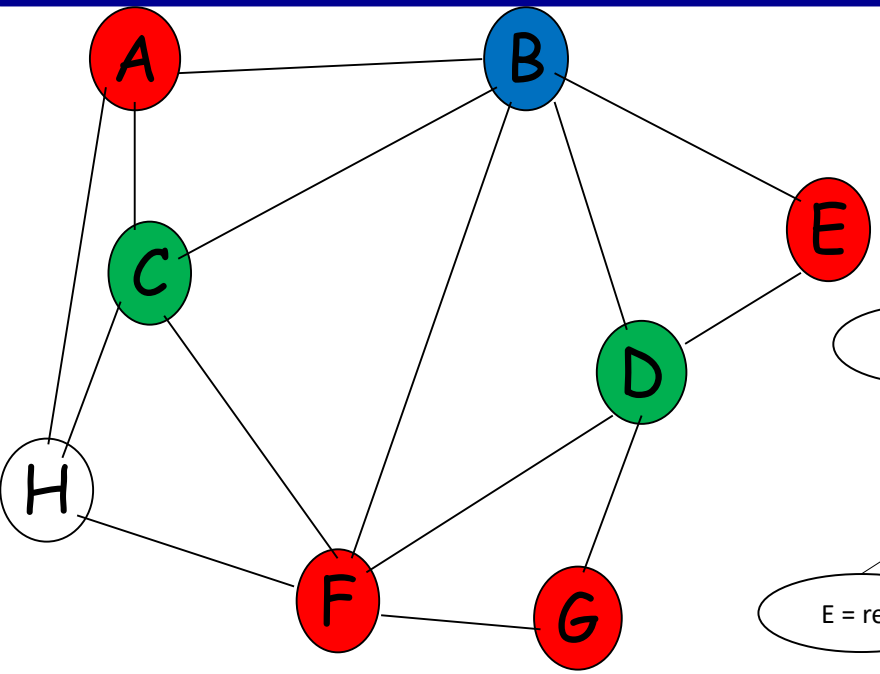
Uninformed BT Algorithm: Map-Coloring Example

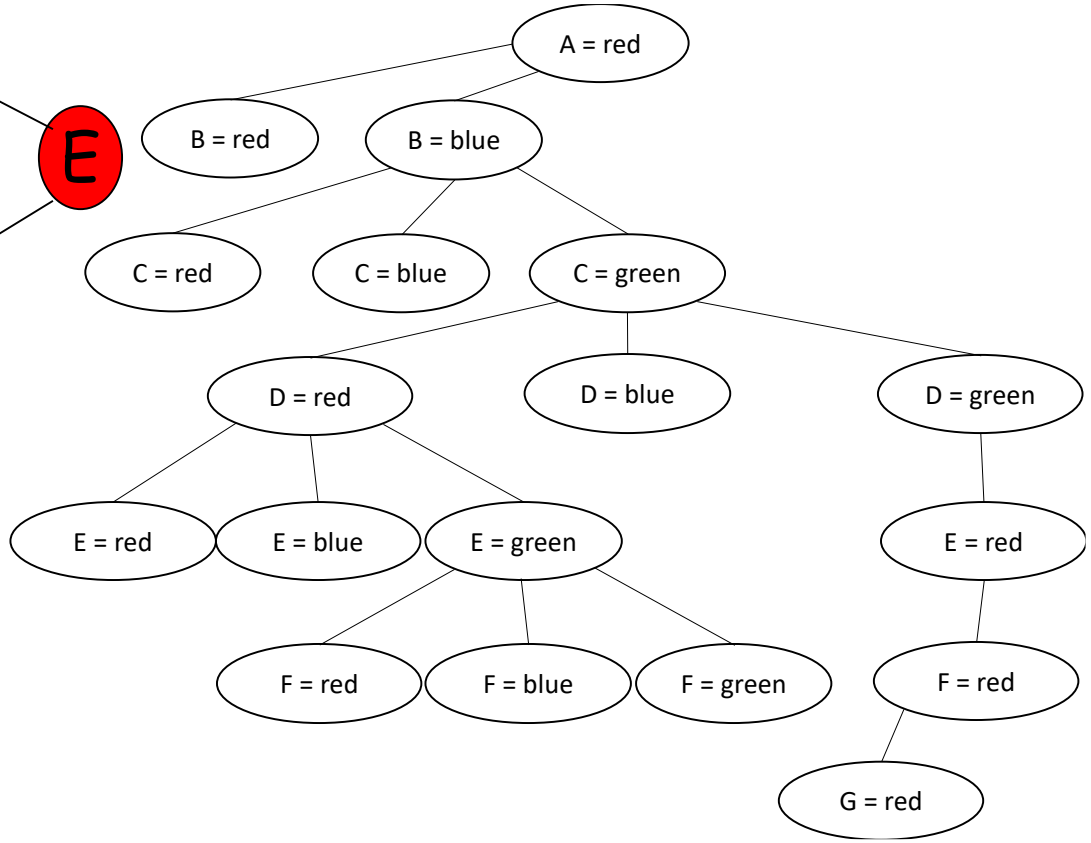




1 = red
2 = blue
3 = green

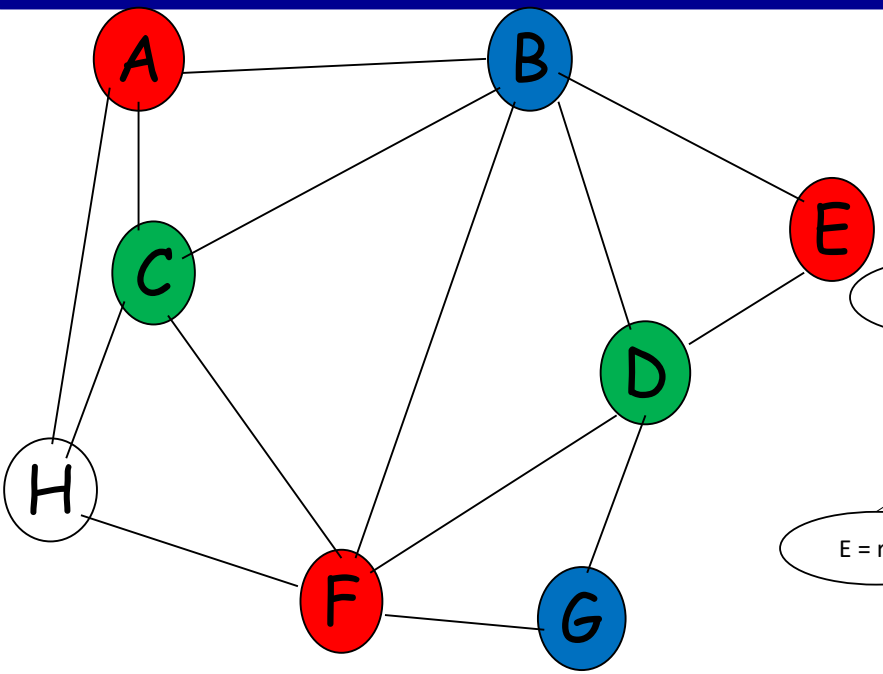
Uninformed BT Algorithm: Map-Coloring Example

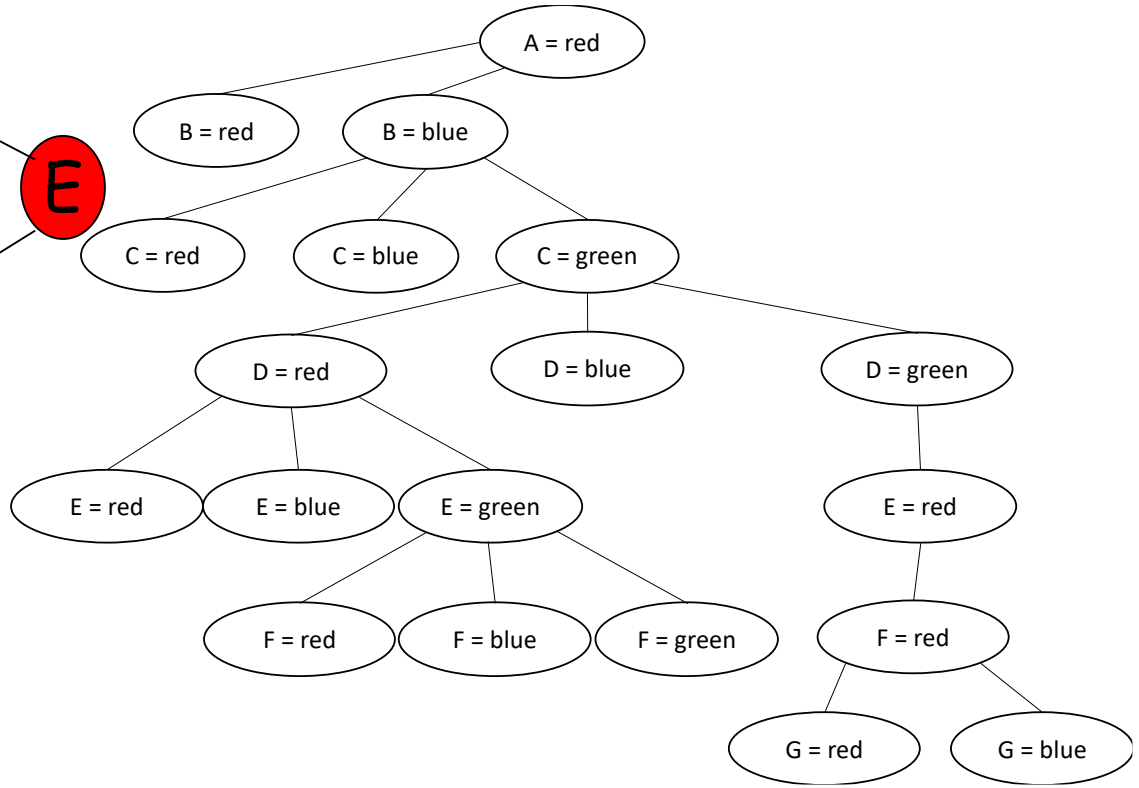




1 = red
2 = blue
3 = green

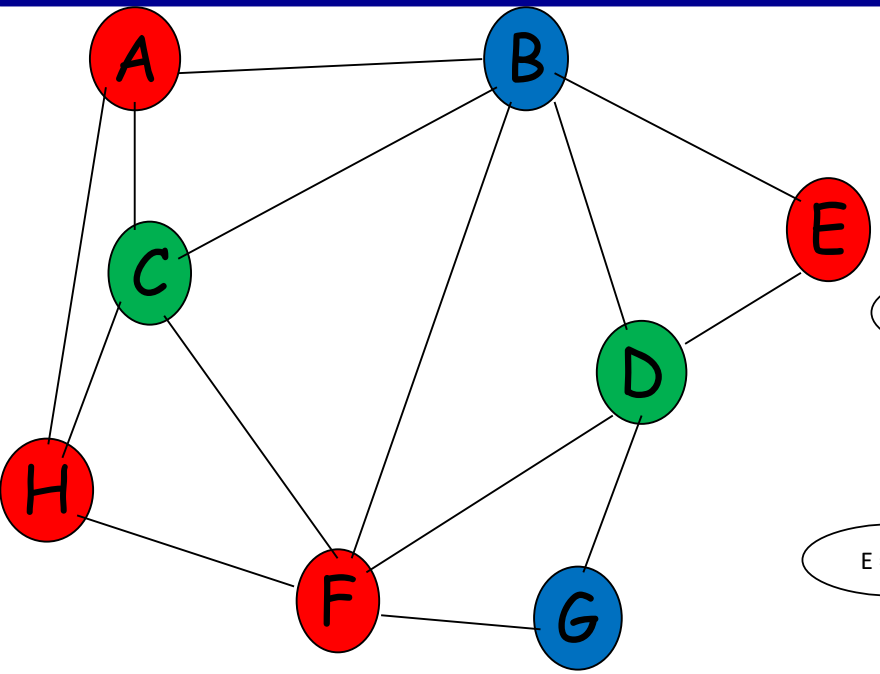
Uninformed BT Algorithm: Map-Coloring Example

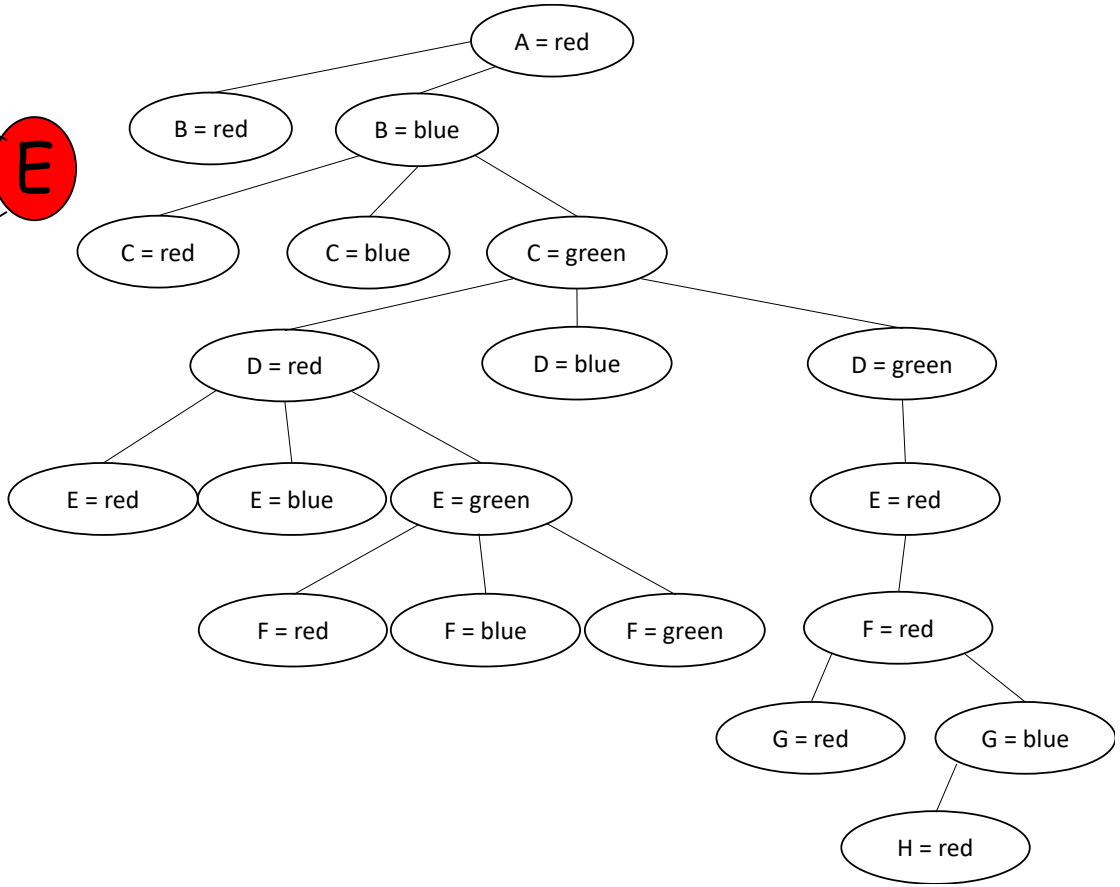




1 = red
2 = blue
3 = green

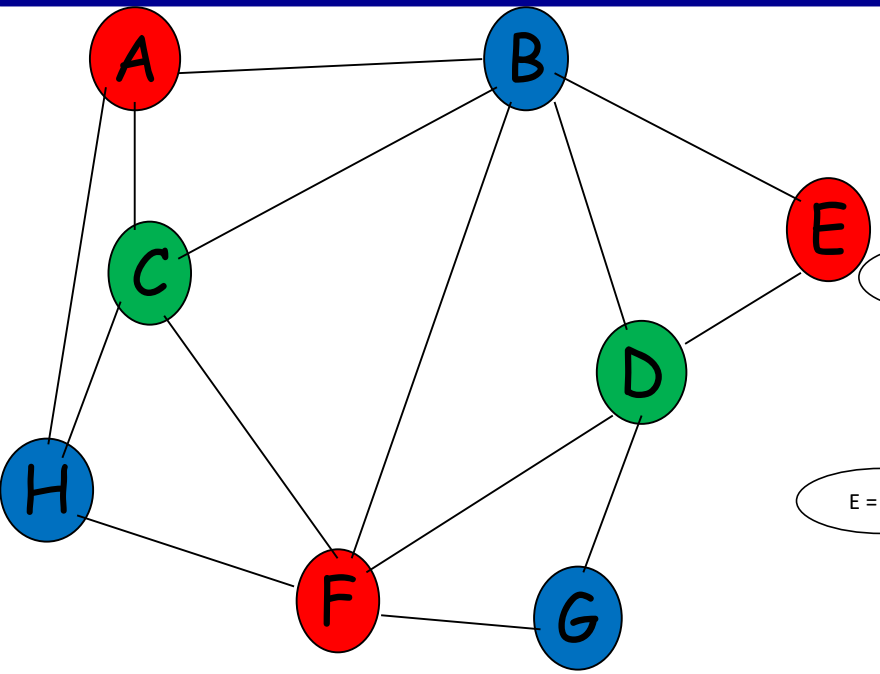
Uninformed BT Algorithm: Map-Coloring Example

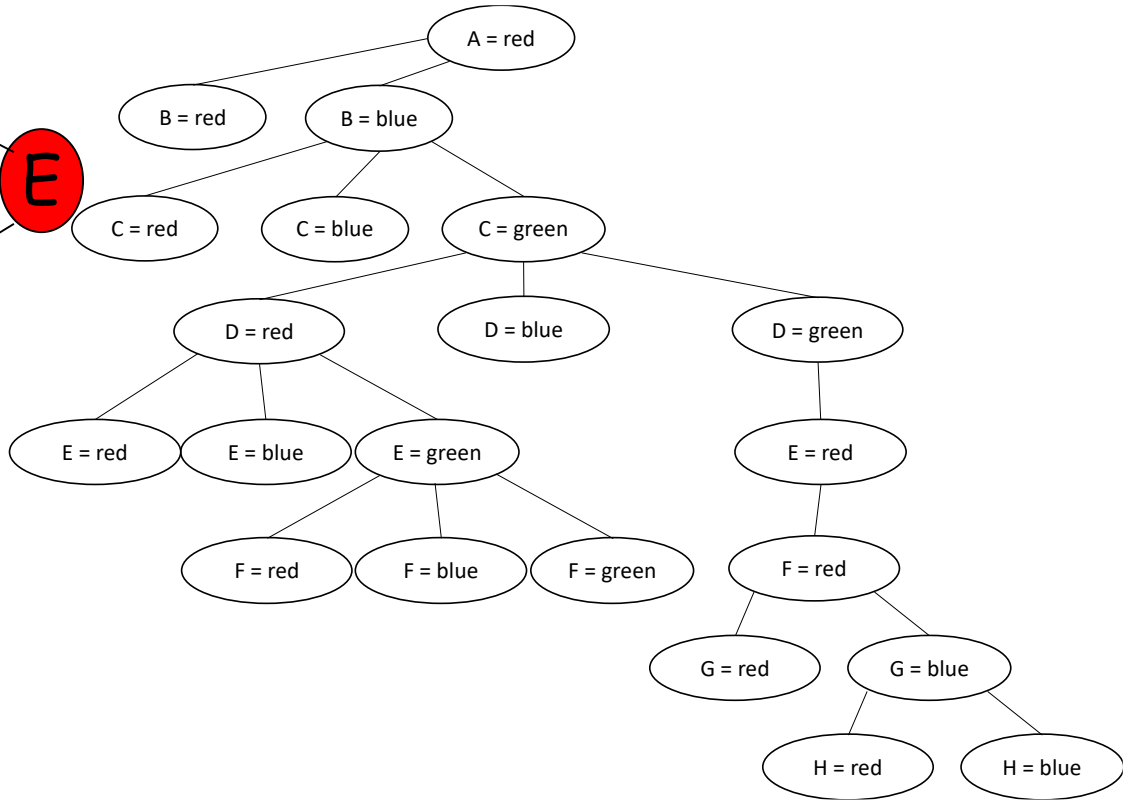




1 = red
2 = blue
3 = green

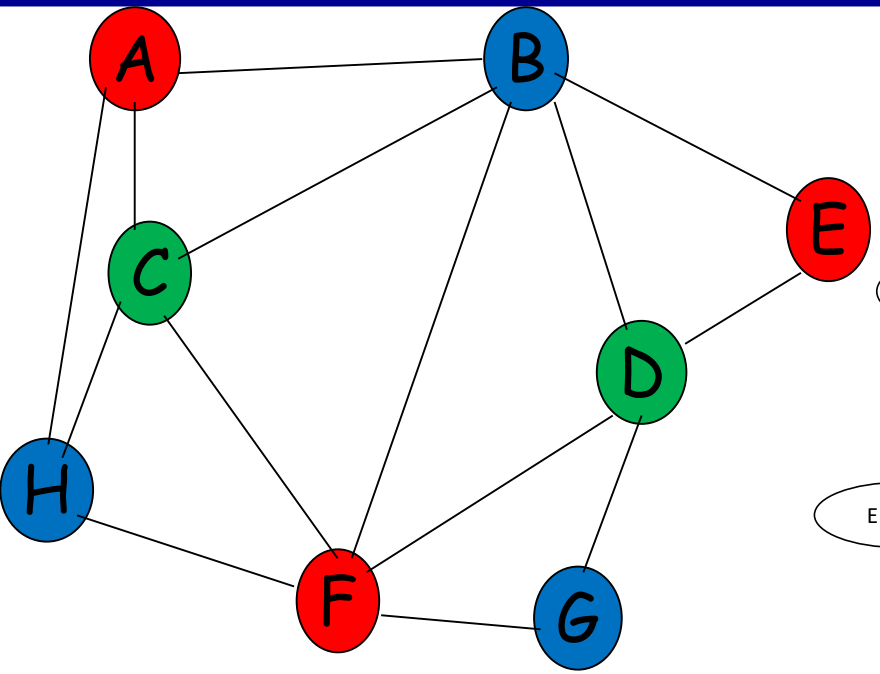
Uninformed BT Algorithm: Map-Coloring Example

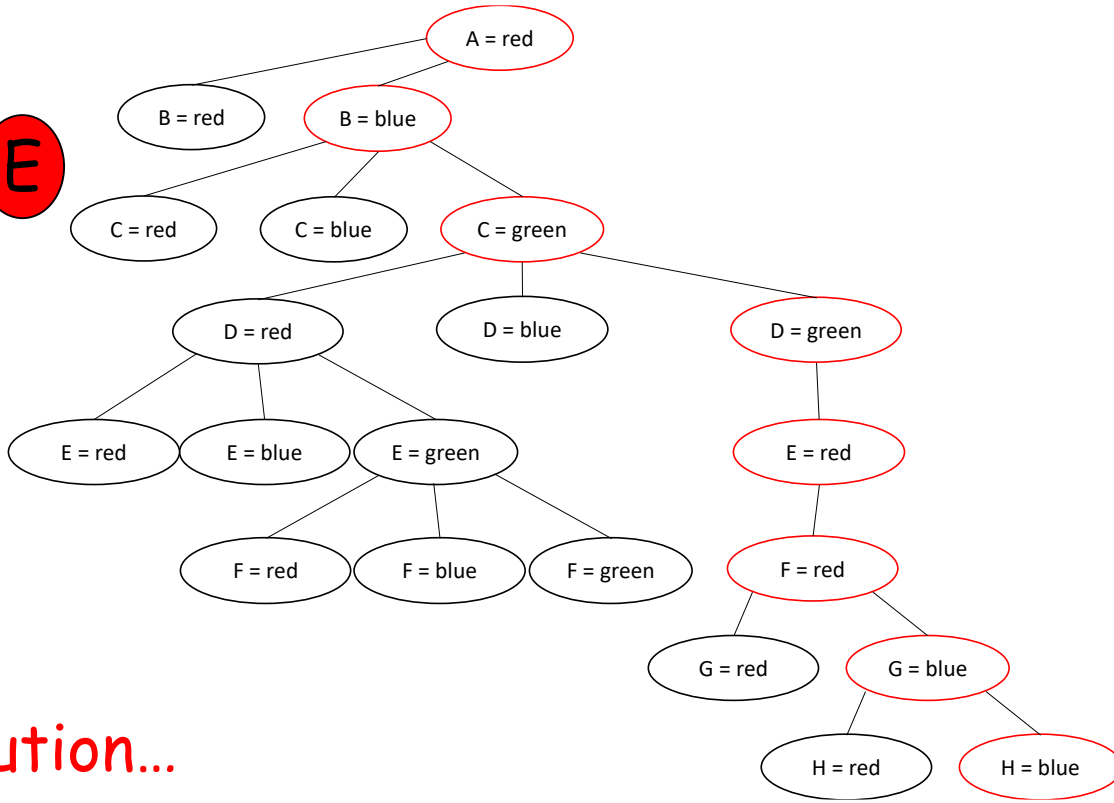




1 = red
2 = blue
3 = green

Uninformed BT Algorithm: Map-Coloring Example





Solution…

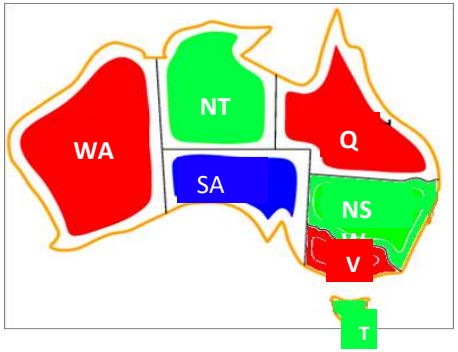
1 = red
2 = blue
3 = green

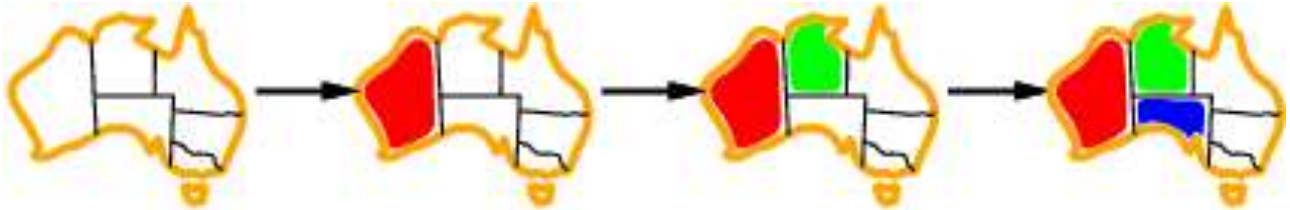
Informed backtrack search

- Improve the backtrack algorithm
- Time complexity of the standard backtrack algorithm is too high
- Knowledge beyond the problem to guide the search
  - At each node of the search tree:
    - Use **constraints** in active way to detect **inconsistent values**.
    - Use **heuristics** to select the **next variable** to instantiate.
    - Use **heuristics** to select **next value** for the selected variable.
- Combining these heuristics makes 1000 queens feasible.

Improving BT: Most constrained variable

- **Most constrained variable:**
  - Choose the variable with the *fewest legal values.*
  - Minimum remaining values (MRV) heuristic.
  - Example: after the assignments for **WA=red** and **NT =green**, there is only one possible value for **SA**, so it makes sense to assign **SA=blue** next rather than assigning **Q.** In fact, after **SA** is assigned, the choices for **Q, NSW**, and **V** are all forced.



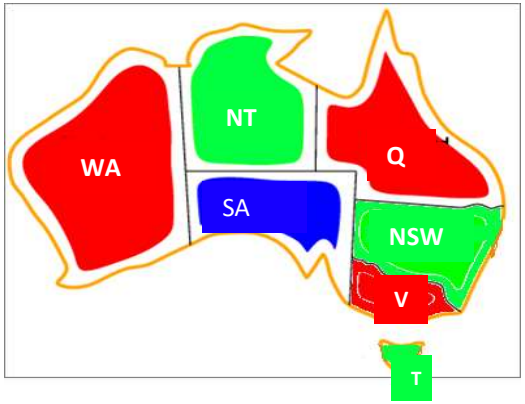


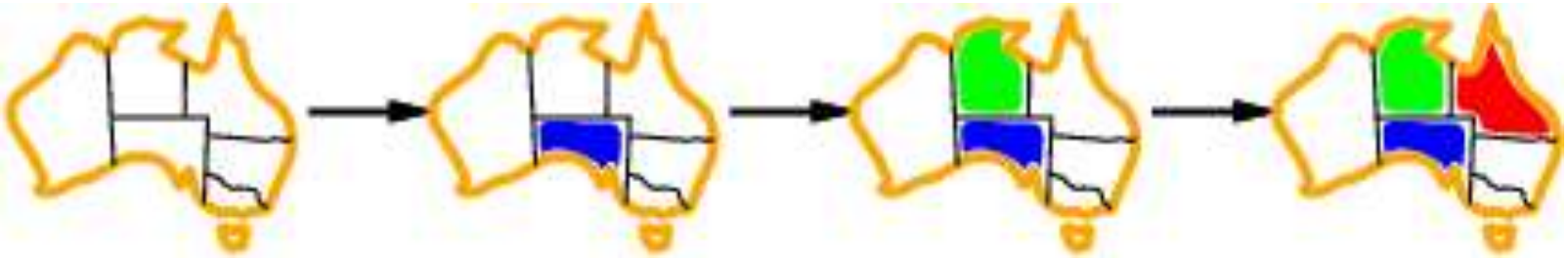
Improving BT: Most constraining variable

- **Most constraining variable**
  - Choose the variable with the most constraints on remaining variables.
  - The MRV heuristic doesn't help at all in choosing the first region to color in Australia, because initially every region has three legal colors.
  - **Tie-breaker** among most constrained variables **(degree heuristic)**
    - It attempts to reduce the branching factor on future choices by selecting the variable that is involved in the largest number of constraints on other unassigned variables.

Improving BT: Most constraining variable

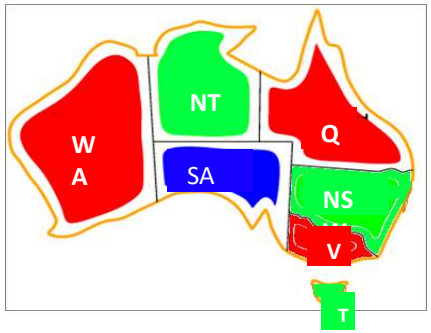
- Example: SA is the variable with highest degree, 5; the other variables have degree 2 or 3, except for T, which has degree 0.

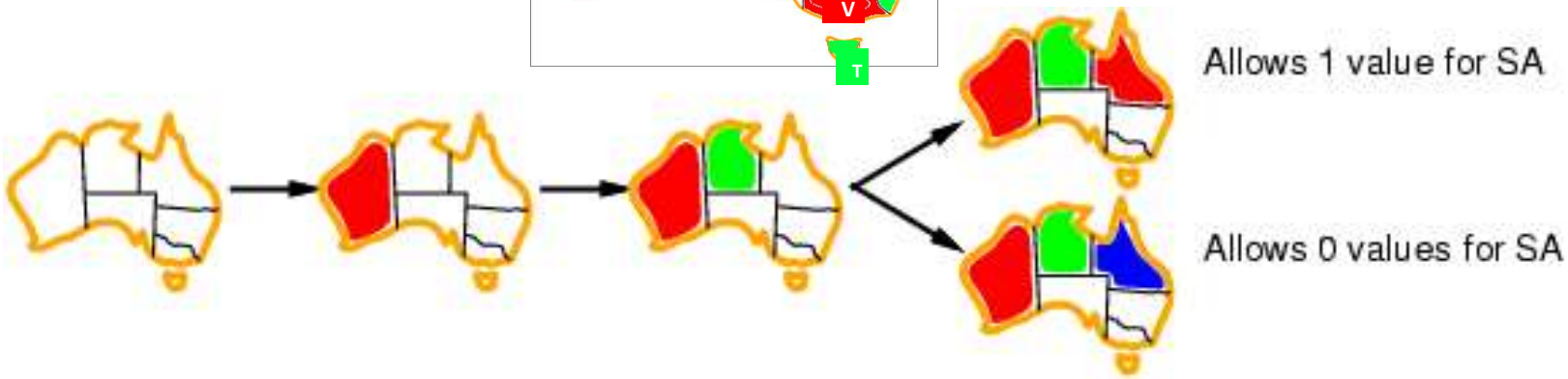




Improving BT: Least constraining value

- **Least constraining value**
  - Given a variable, choose the least constraining value.
    - It prefers the value that rules out the fewest choices for the neighboring variables.
  - Example: If WA=red and NT =green, and that the next choice is for Q. Blue would be a bad choice because it eliminates the last legal value left for Q's neighbor, SA.





Constraint propagation: inference in CSPs

- Constraint propagation propagates arc consistency on the graph.
  - Simplest form of propagation makes each arc consistent.
- Arc consistency aims to detect failure early
  - Using the constraints to reduce the number of legal values for a variable, which in turn can reduce the legal values for another variable, and so on.
- Can be run as a preprocessor or after each assignment.
- Sometimes this preprocessing can solve the whole problem, so no search is required at all.

Constraint propagation: inference in CSPs

- **Improving BT** (or any CSP search algorithm):
  - Transforming the initial CSP to an equivalent CSP easier than the original one.

- **Objective:** reduce the size of the CSP, by :
  - Removing inconsistent values.
  - In some cases, the CSP is solved
    - A solution is found
    - An empty domain is found $\Rightarrow$ no solution.

- Achieved before (e.g., AC-3 algorithm) or during (e.g., Forward checking algorithm) searching.

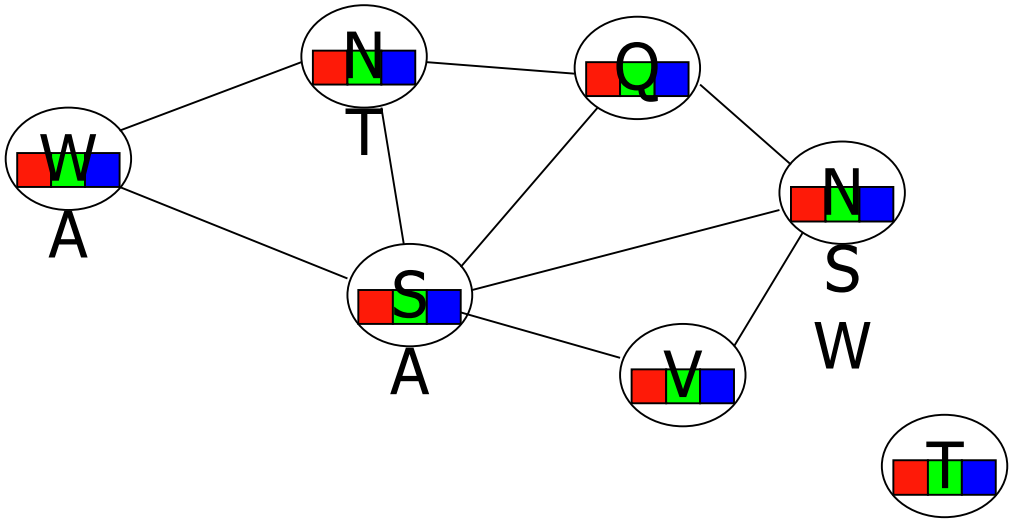
Arc consistency (AC)

- A variable in a CSP is arc-consistent if every value in its domain satisfies the variable's binary constraints.
- $X \rightarrow Y$ is consistent iff for every value x of X there is some allowed y

- E.g., $Y = X^2$ where the domain of both X and Y is the set of digits.
  - We can write this constraint explicitly as (X, Y), {(0, 0), (1, 1), (2, 4), (3, 9))} .
  - To make X arc-consistent with respect to Y, we reduce X's domain to {0, 1, 2, 3}.
  - If we also make Y arc-consistent with respect to X, then Y 's domain becomes {0, 1, 4, 9} and the whole CSP is arc-consistent.

Arc consistency (AC)

- E.g., arc consistency can do nothing for the Australia map-coloring problem.
  - Consider the following inequality constraint on (SA,WA): {(red , green), (red , blue), (green, red), (green, blue), (blue, red), (blue, green)}
  - No matter what value you choose for SA (or for WA), there is a valid value for the other variable.
  - applying arc consistency has no effect on the domains of either variable.



Arc consistency (AC)

- Can be run as a preprocessing before search starts or after each assignment
  - AC must be run repeatedly until no inconsistency remains

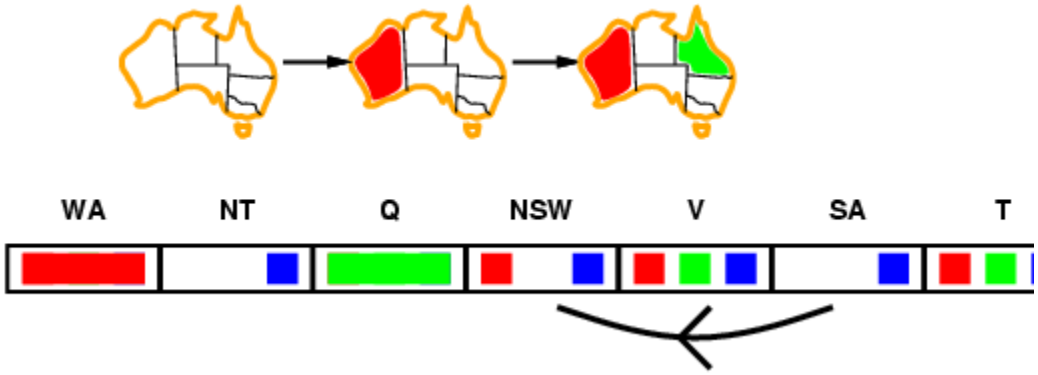
- Trade-off
  - Requires some overhead to do, but generally more effective than direct search
    - It can eliminate large (inconsistent) parts of the state space more effectively than search can

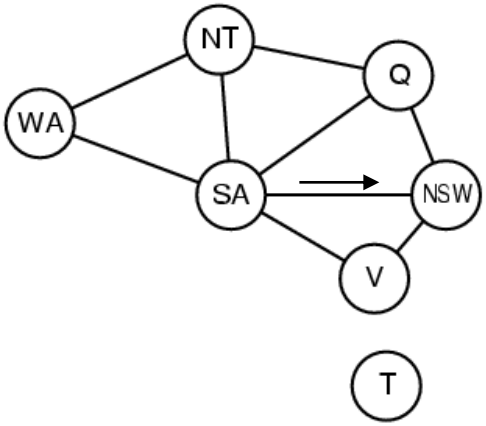
- Need a systematic method for arc-checking
  - If *X* loses a value, neighbors of *X* need to be rechecked:
    - **incoming** arcs can become inconsistent again
    - **outgoing** arcs will stay consistent

Maintaining Arc consistency

- Simplest form of propagation makes each arc consistent
- $X \rightarrow Y$ is consistent iff

  for every value x of X there is some allowed y



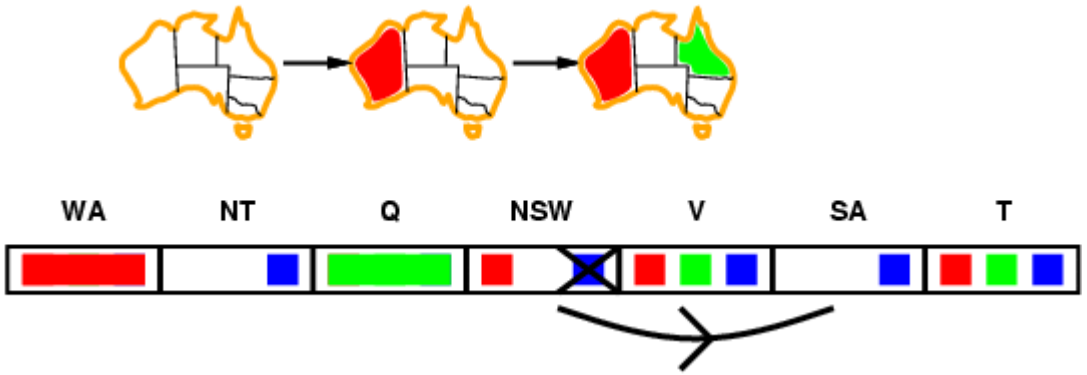


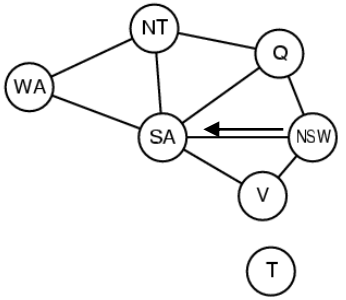
constraint propagation propagates arc consistency on the graph.

Maintaining Arc consistency

- Simplest form of propagation makes each arc consistent
- $X \rightarrow Y$ is consistent iff

  for every value x of X there is some allowed y

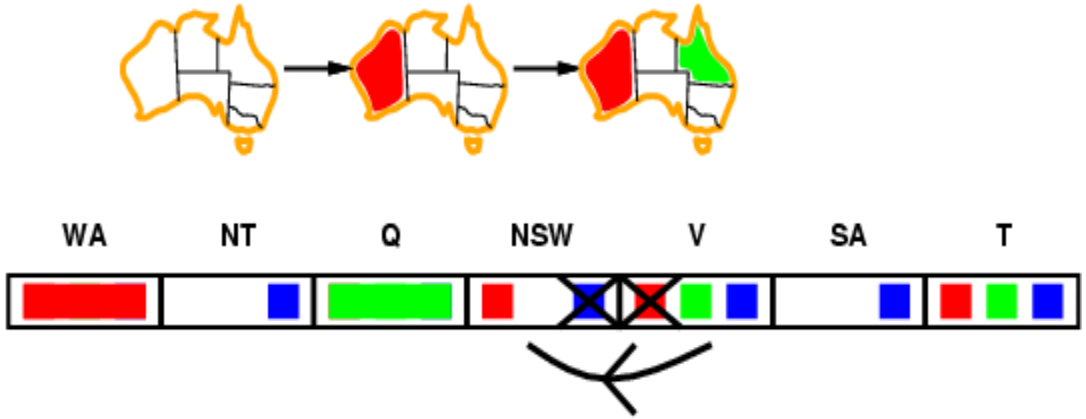


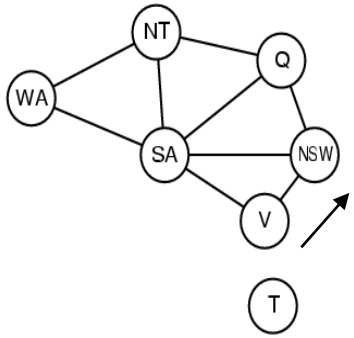


Maintaining Arc consistency

- Simplest form of propagation makes each arc consistent
- $X \rightarrow Y$ is consistent iff

  for every value x of X there is some allowed y



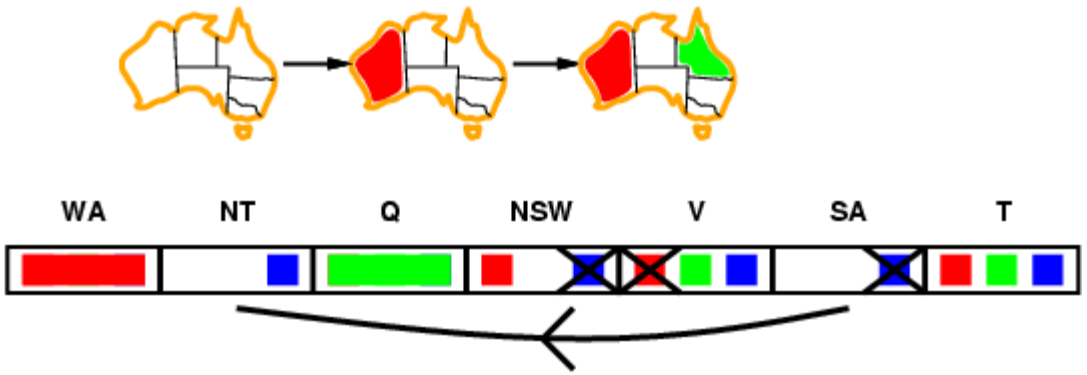


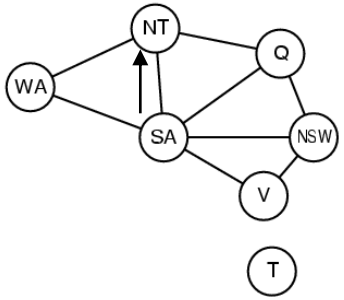
- If X loses a value, neighbors of X need to be rechecked

Maintaining Arc consistency

- Simplest form of propagation makes each arc consistent
- $X \rightarrow Y$ is consistent iff

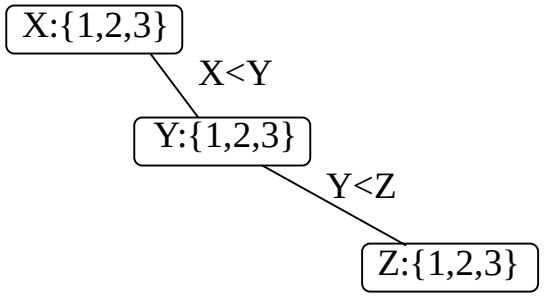
  for every value x of X there is some allowed y



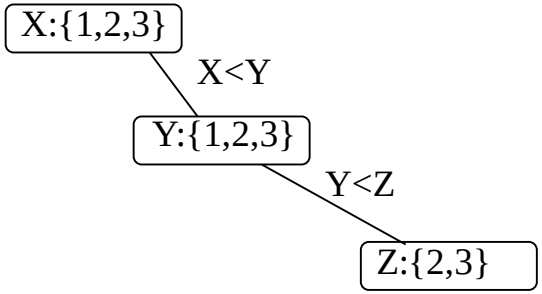


- If X loses a value, neighbors of X need to be rechecked
- Arc consistency aims to detect failure early
- Can be run as a preprocessor or after each assignment

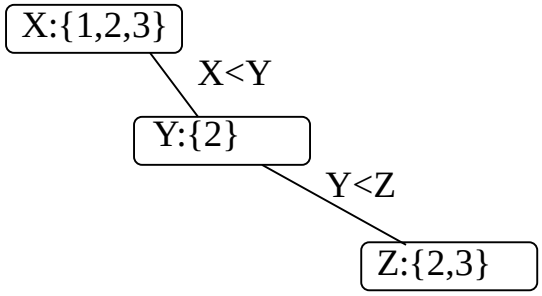
Enforcing Arc Consistency: An example



- When we enforce arc-consistency on the top left CSP (shown as a constraint graph), we get the CSP shown in the bottom. Notice how the domains have reduced.

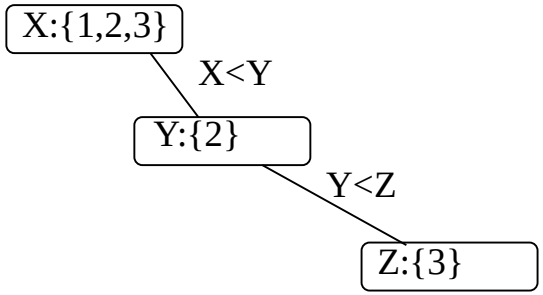


- Suppose we start from **Z**. If **Z=1**, then **Y** can't have any valid values. So, we **remove** **1** from **Z**'s domain. If **Z=2,** or **3** then **Y** can have a valid value (since **Y** can be **1** or **2**).

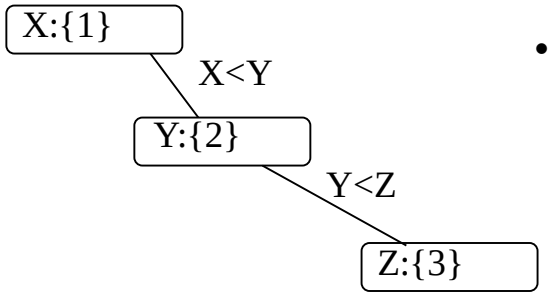


- Now we go to **Y**. If **Y=1**, then **X** can't have any value. So, we **remove** **1** from **Y**'s domain. If **Y=3**, then **Z** can't have any value. So, we **remove** **3** from **Y**'s domain. So **Y** has just **2** in its domain!

Enforcing Arc Consistency: An example



- Now notice that **Y**'s domain has changed. So, we **re-consider Z** (since anytime **Y**'s domain changes, it is possible that **Z**'s domain gets affected). Sure enough, **Z=2** no longer works since **Y** can only be **2** and so it can't take any value if **Z=2.** So, we **remove 2** also from **Z**'s domain. So, **Z**'s domain is now just **3**!



- Now, we go to **X. X** can't be **2** or **3** (since for either of those values, **Y** will not have a value. So, **X** has domain **1**!

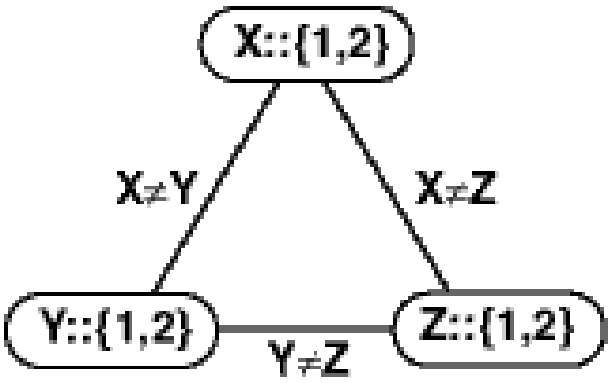
- Notice that in this case, arc-consistency SOLVES the problem, since **X,Y** and **Z** have exactly **1** value each and that is the only possible solution. **This is not always the case**.

More on arc-consistency

- Arc-consistency doesn't always imply that the CSP has a solution or that there is no search required.

- In the previous example, if each variable had domains 1,2,3,4, then at the end of enforcing arc-consistency, each variable will still have 2 values in its domain—thus necessitating search.

- Here is another example which shows that the search may find that there is **no solution** for the CSP, even though it is arc-consistent.



Here is a binary CSP that *is* arc-consistent but has no Solution.

Arc consistency algorithm (AC-3)

- The most popular algorithm for arc consistency is called AC-3.

function AC-3(*csp*) **returns** false if an inconsistency is found and true otherwise
  inputs: *csp*, a binary CSP with components (X, D, C)
  local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

  while *queue* is not empty **do**
    $(X_i, X_j) \leftarrow$ REMOVE-FIRST(*queue*)
    if REVISE(*csp*, X_i, X_j) **then**
      if size of $D_i = 0$ **then return** *false*
      for each X_k **in** X_i.NEIGHBORS - $\{X_j\}$ **do**
        add (X_k, X_i) to *queue*
  return *true*

---

function REVISE(*csp*, X_i, X_j) **returns** true iff we revise the domain of X_i
  revised $\leftarrow$ *false*
  for each x **in** D_i **do**
    if no value y in D_j allows (x,y) to satisfy the constraint between X_i and X_j **then**
      delete x from D_i
      revised $\leftarrow$ *true*
  return *revised*

Complexity of AC-3

- A **binary CSP** has at most n^2 arcs
- Each arc can be inserted in the queue d times (worst case)
  - (X, Y): only d values of X to delete
- Consistency of an arc can be checked in $O(d^2)$ time
- Complexity is $O(n^2 d^3)$

AC-3: Example

See CSP Tutorial, question 2.

Forward checking

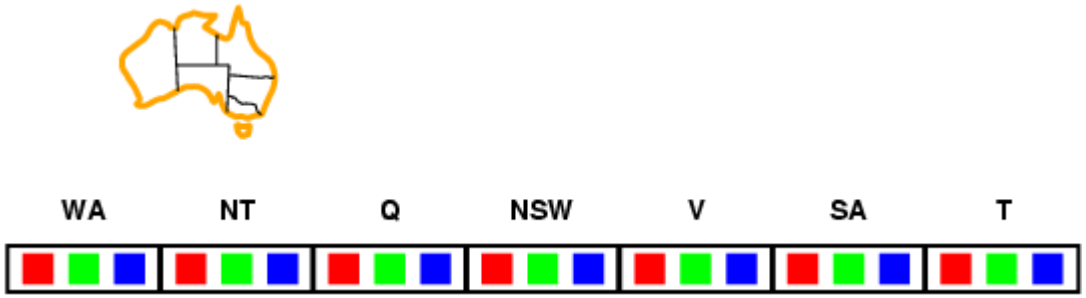
- The AC-3 algorithm infers reductions before beginning the search.
- Forward Checking infers them **during** the search.
- Forward Checking assigns variable X, say
  - Looks at each unassigned variable, Y say, connected to X and delete from Y's domain any value inconsistent with X's assignment
  - Eliminates branching on certain variables by *propagating information*
  - If forward checking detects a dead end, algorithm will backtrack immediately.

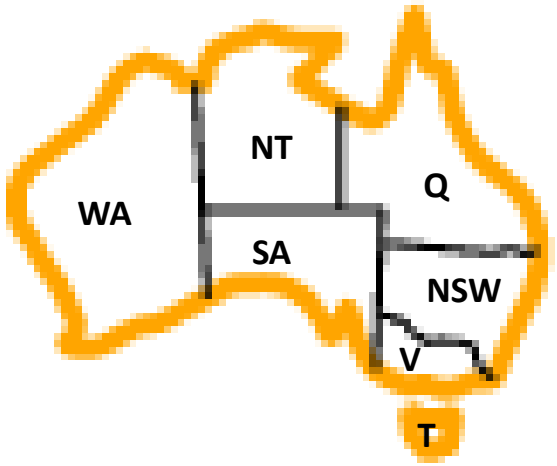
Forward checking

- Because Forward Checking only does arc-consistency inferences, there is no reason to do Forward Checking if arc-consistency was already done as a preprocessing step.
- For many problems, the search will be more effective if we combine the MRV heuristic with forward checking.

Forward checking: Map coloring problem

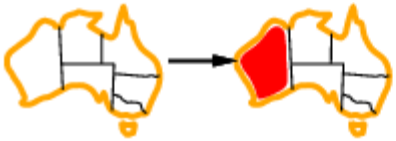
- **Idea:**
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values

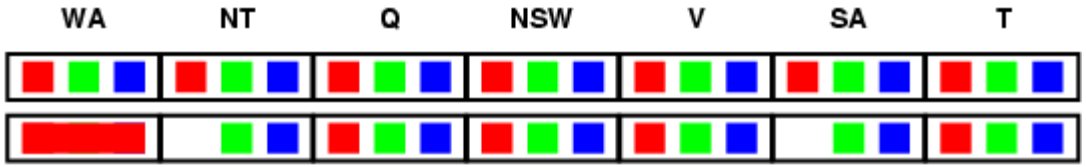


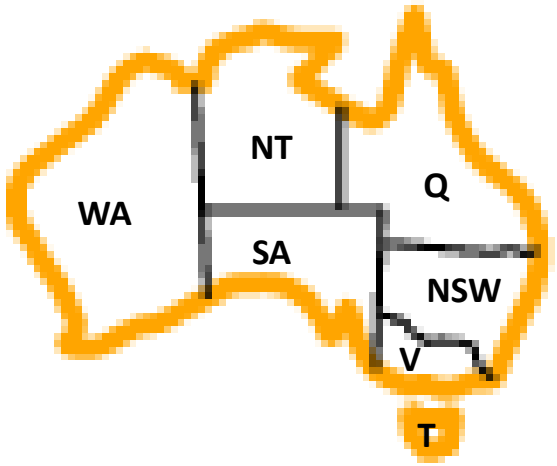


Forward checking: Map coloring problem

- **Idea:**
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values

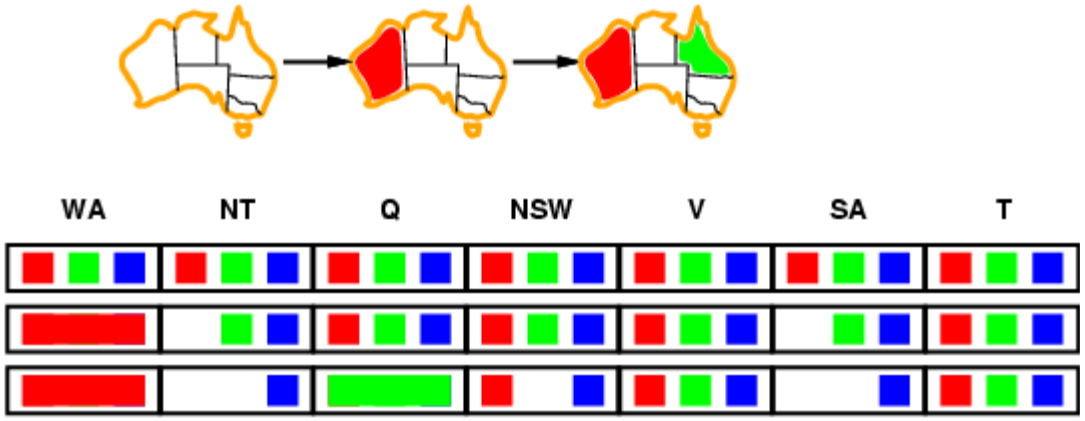


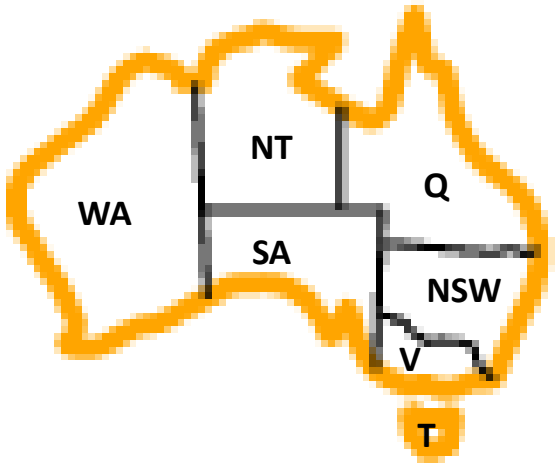




Forward checking: Map coloring problem

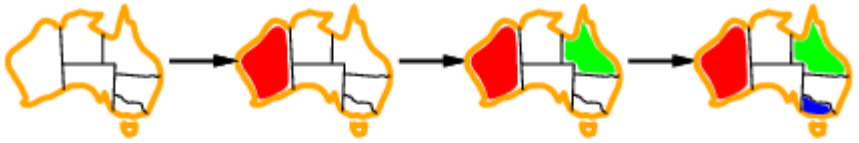
- **Idea:**
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values





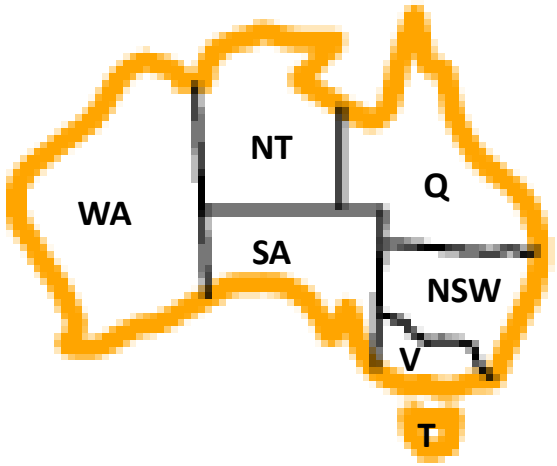
Forward checking: Map coloring problem

- **Idea:**
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values

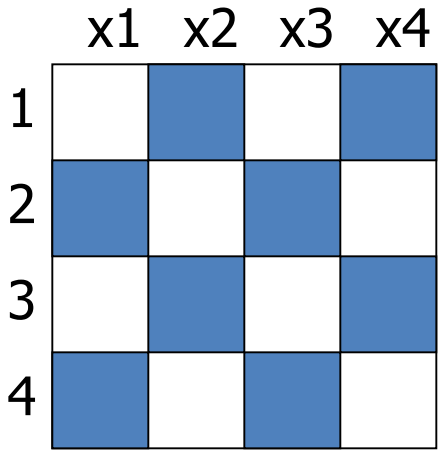


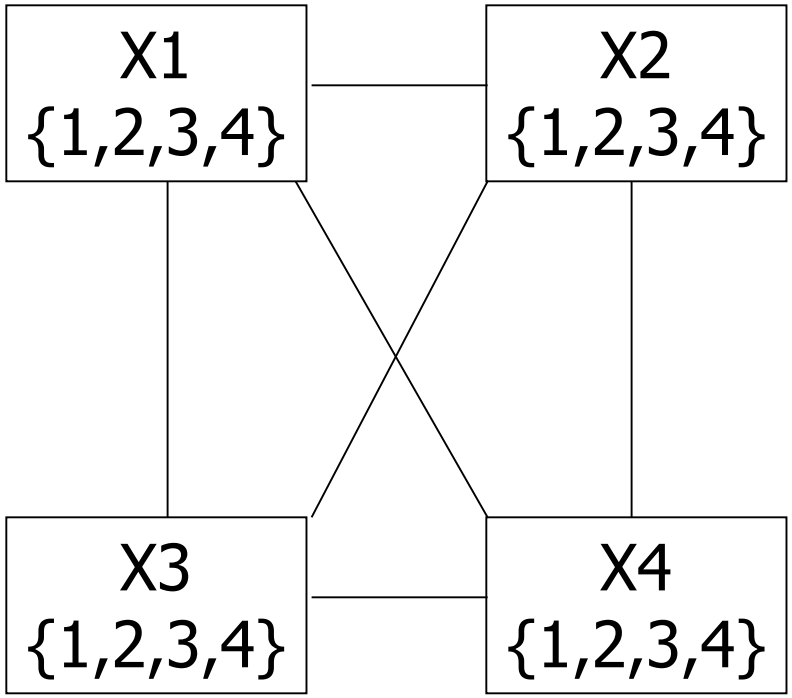


Dead End



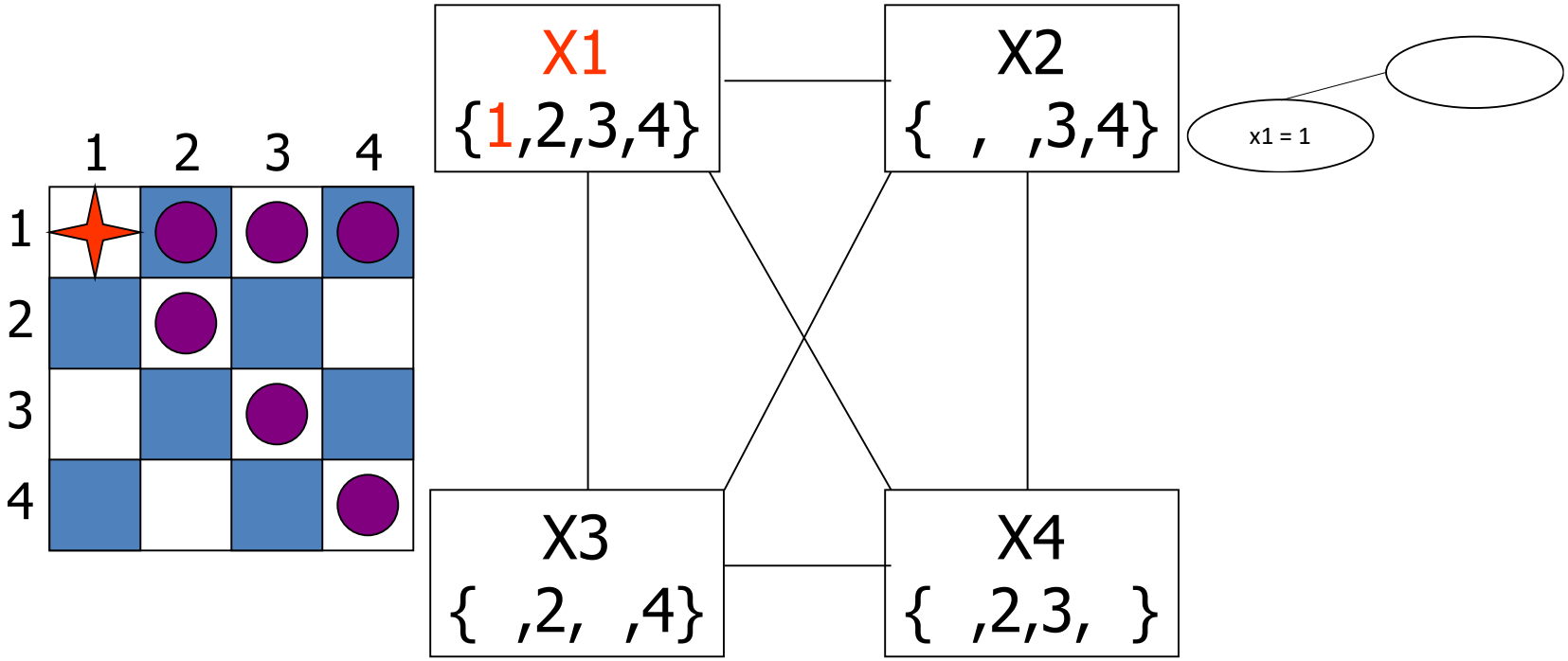
Forward Checking: 4-Queens Problem



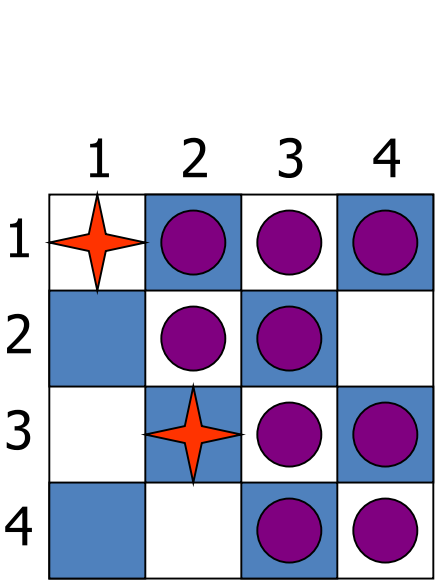


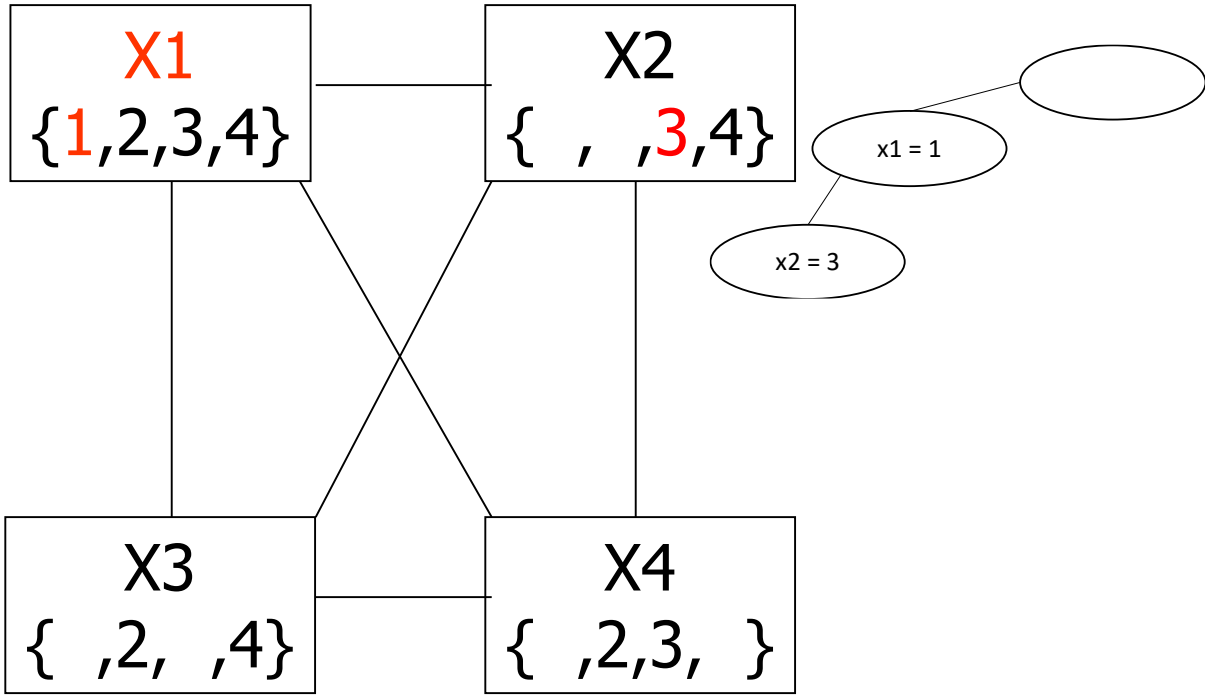
[4-Queens slides copied from B.J. Dorr CMSC 421 course on AI]

Forward Checking: 4-Queens Problem

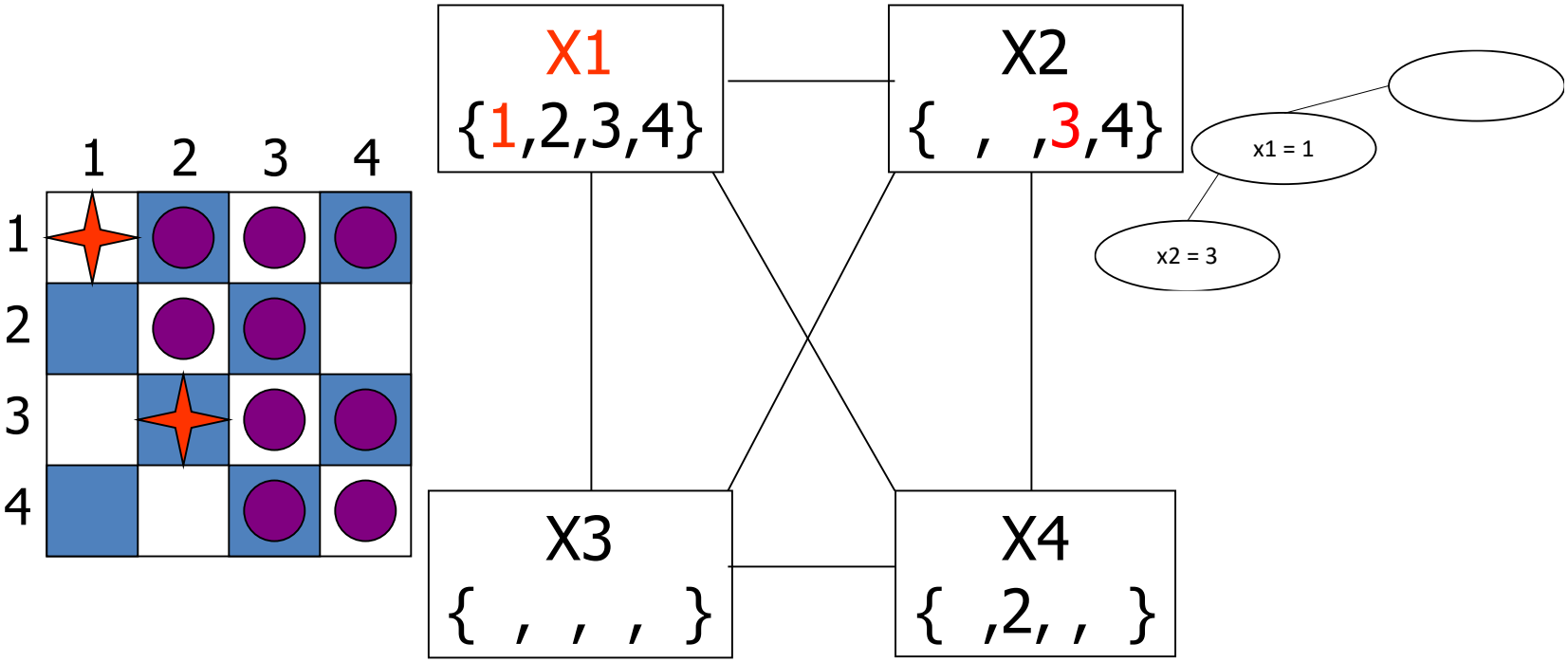


Forward Checking: 4-Queens Problem



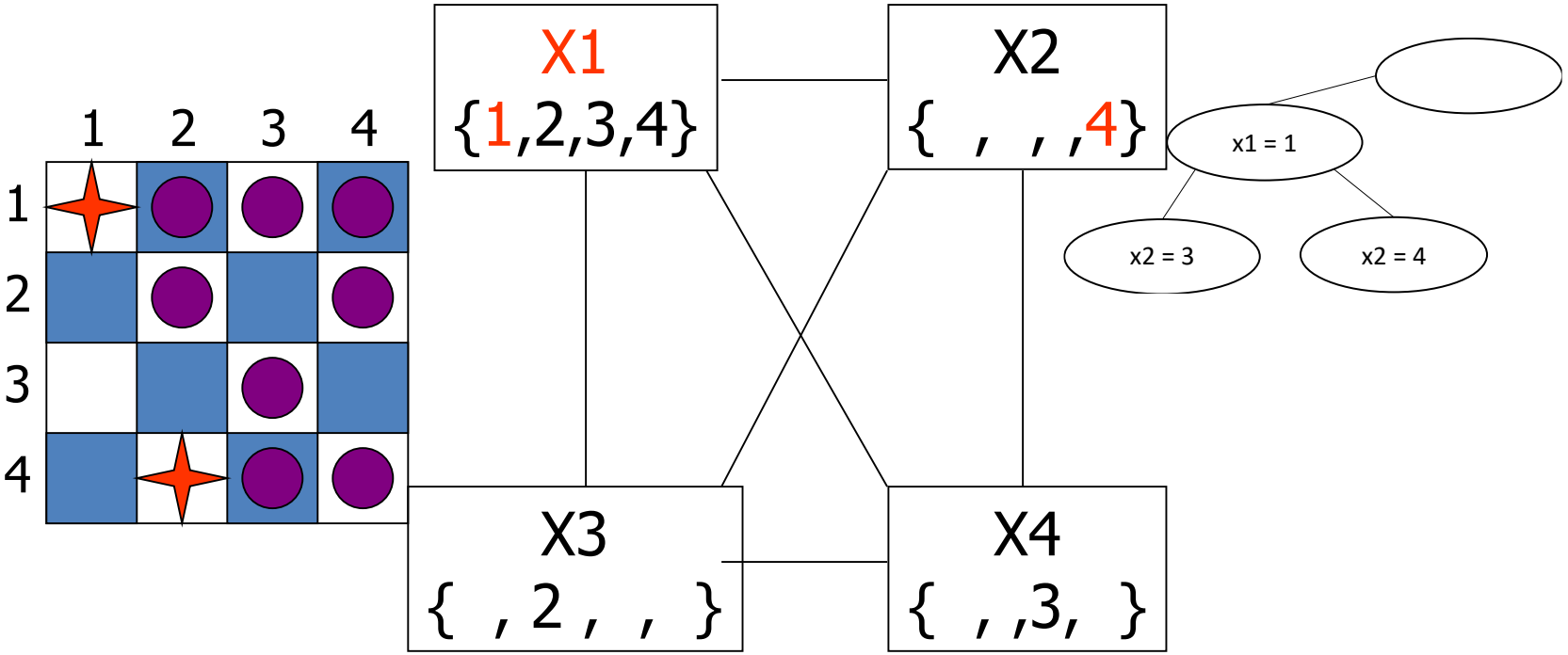


Forward Checking: 4-Queens Problem

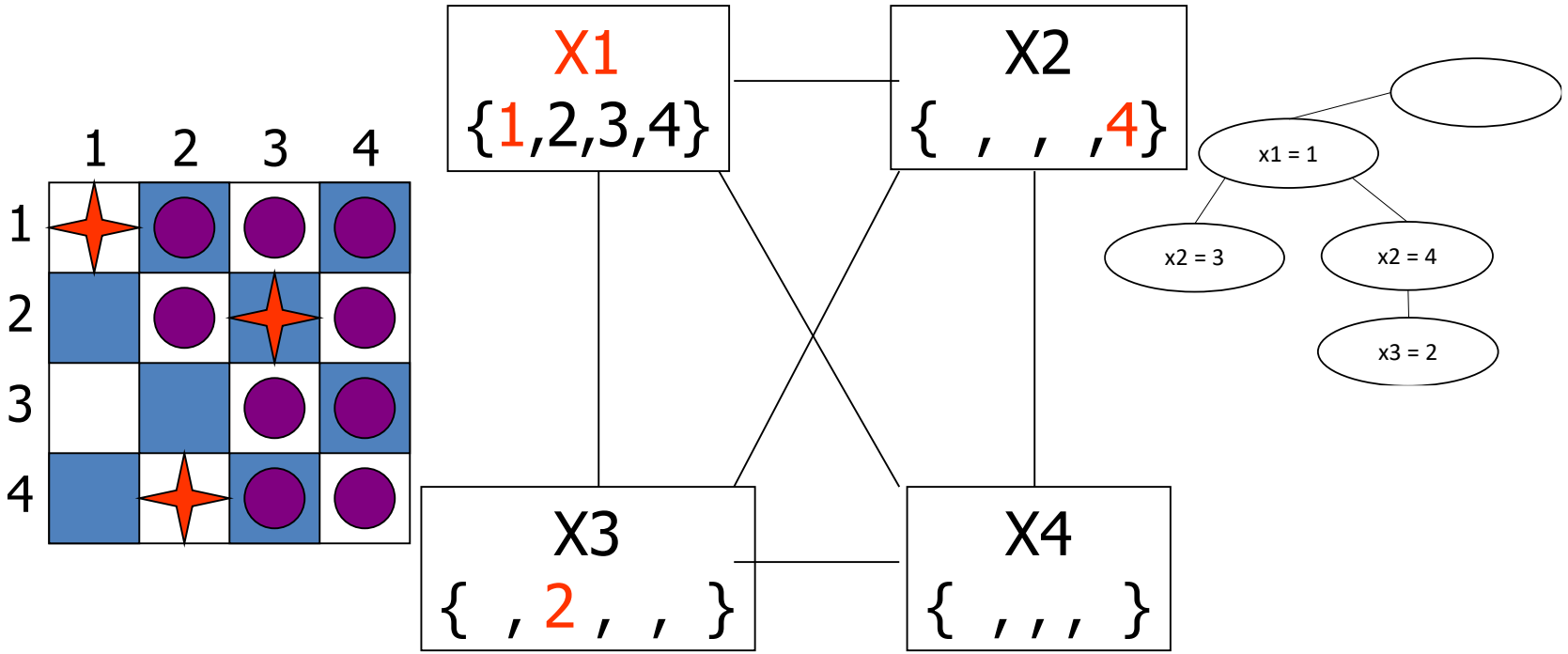


Dead End → Backtrack

Forward Checking: 4-Queens Problem

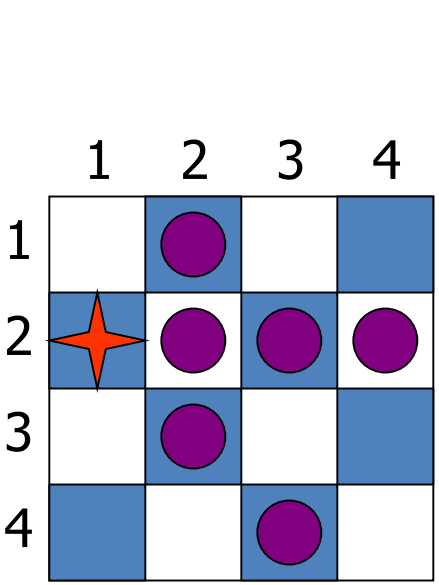


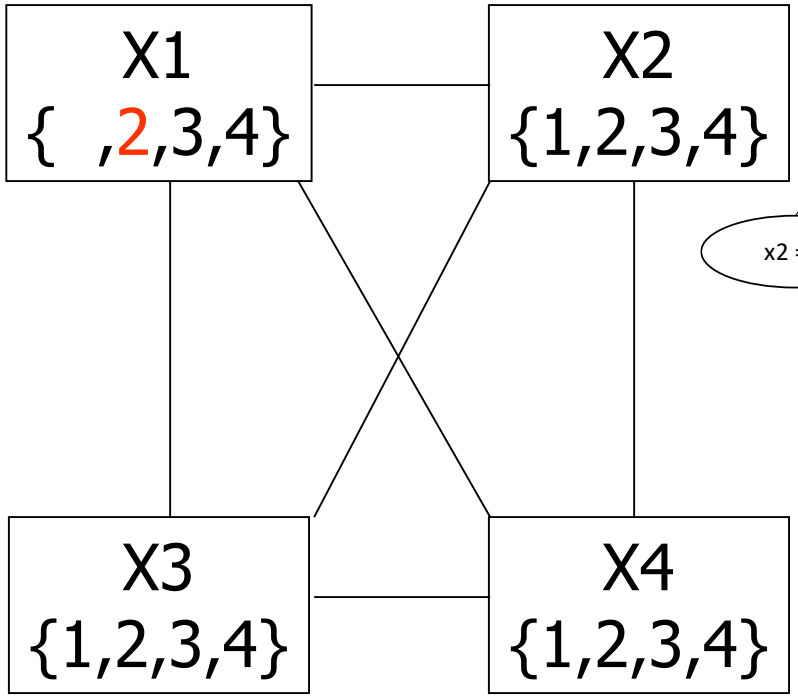
Forward Checking: 4-Queens Problem

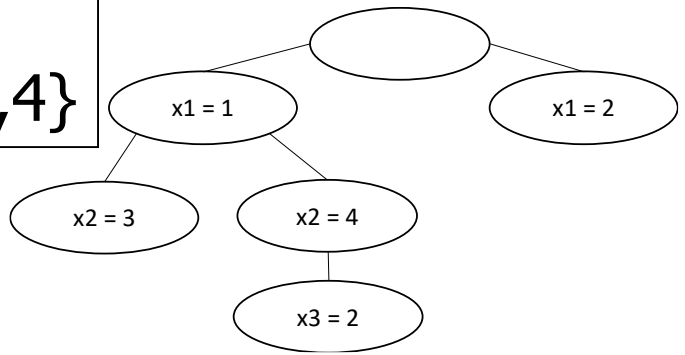


Dead End → Backtrack

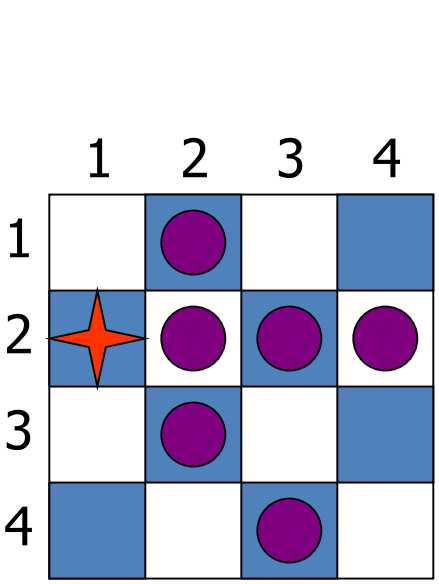
Forward Checking: 4-Queens Problem

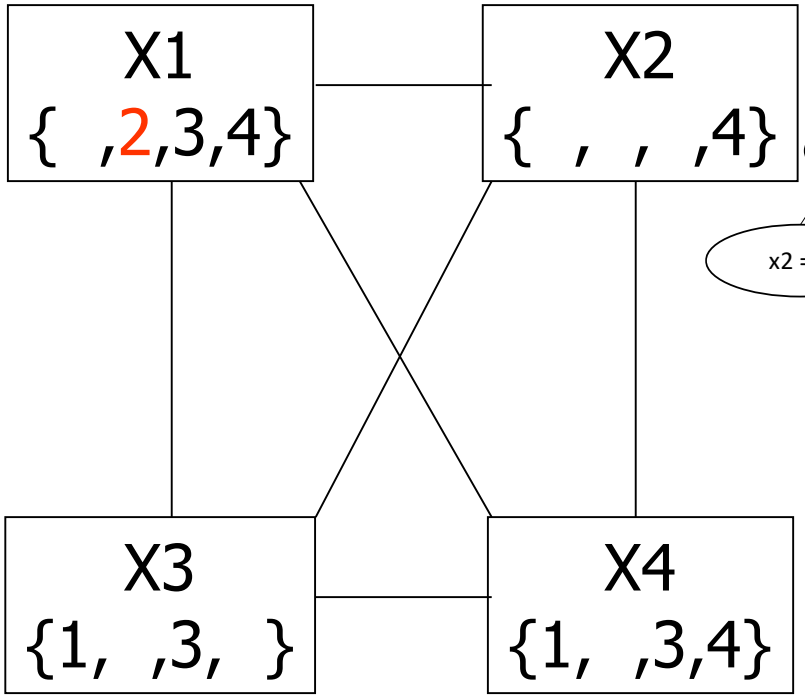


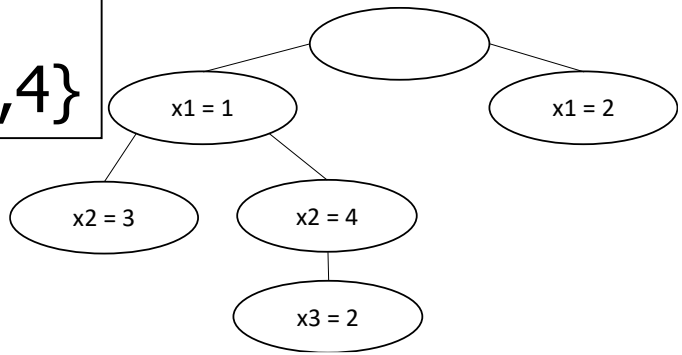




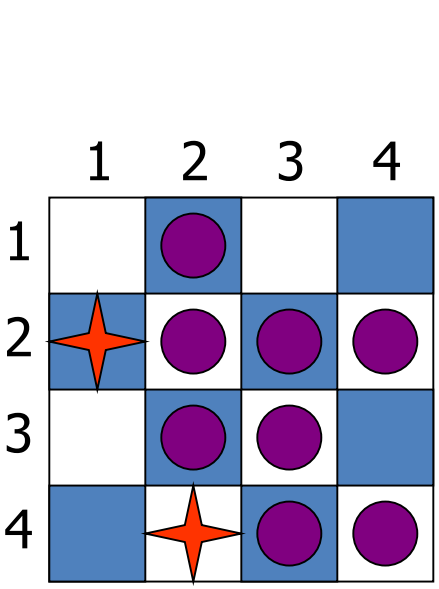
Forward Checking: 4-Queens Problem

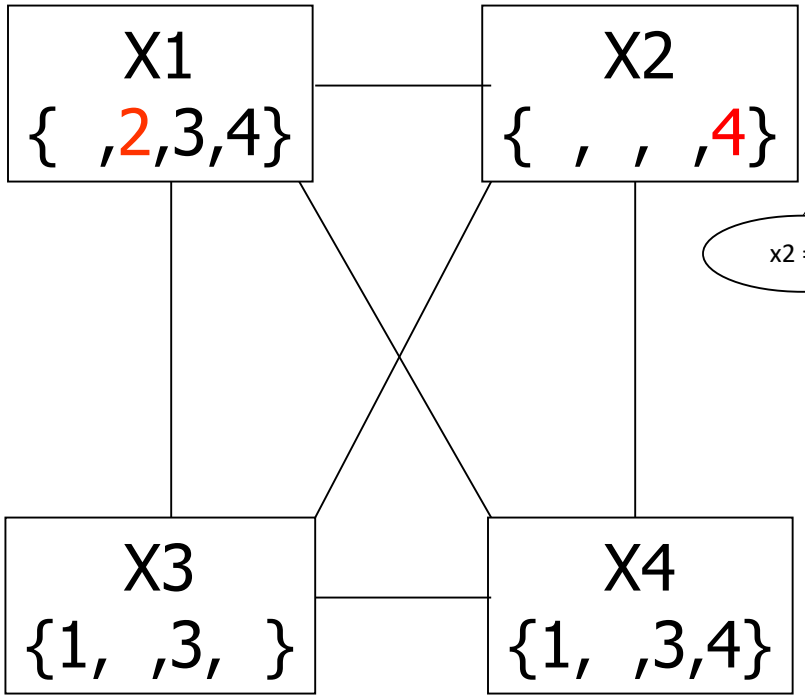


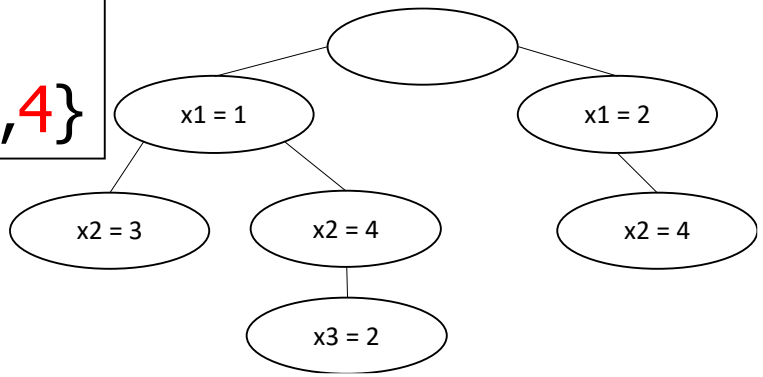




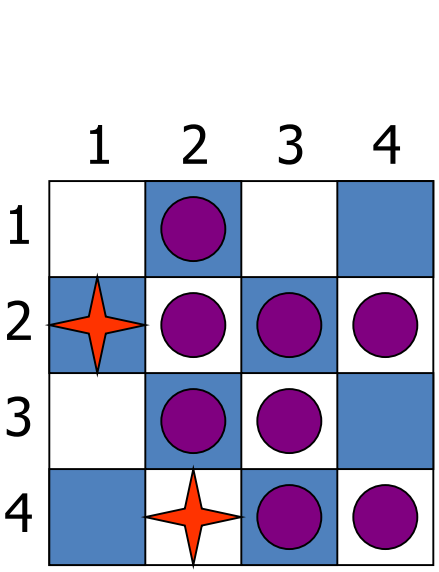
Forward Checking: 4-Queens Problem

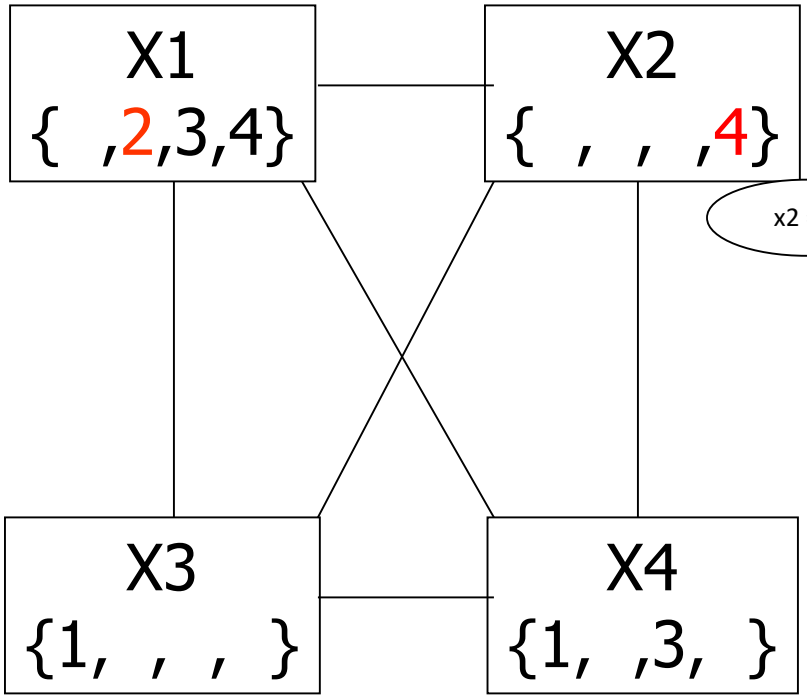


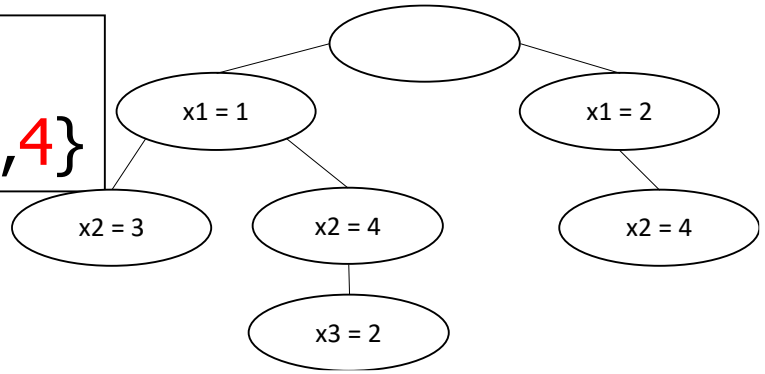




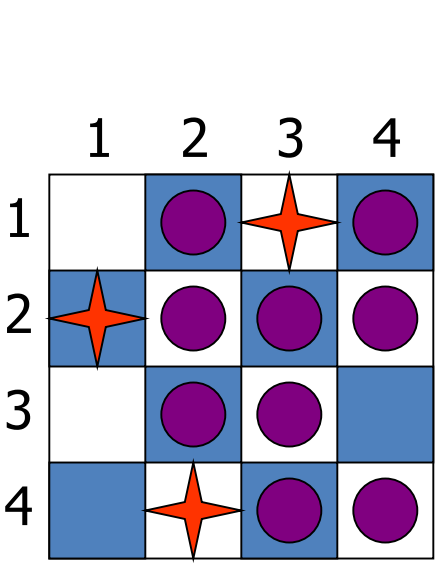
Forward Checking: 4-Queens Problem

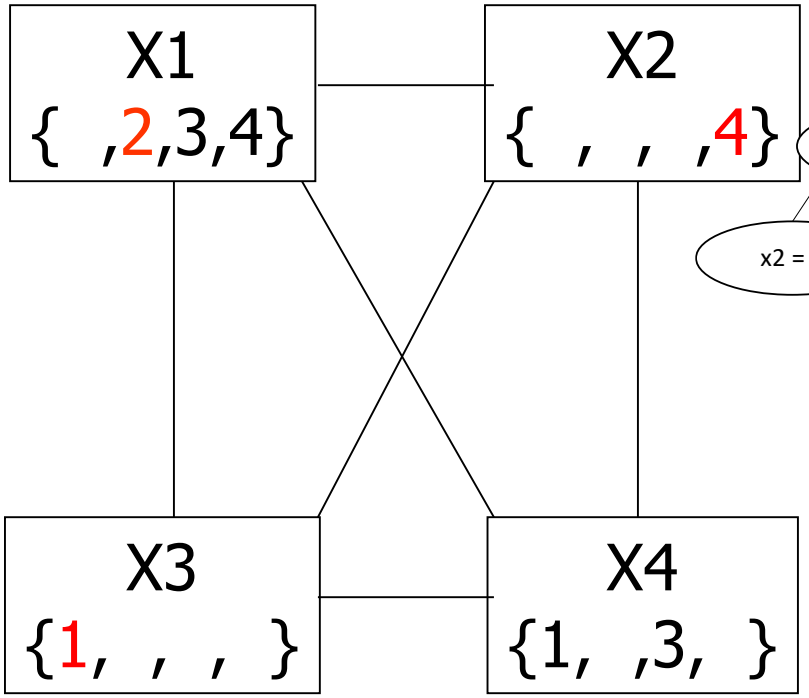


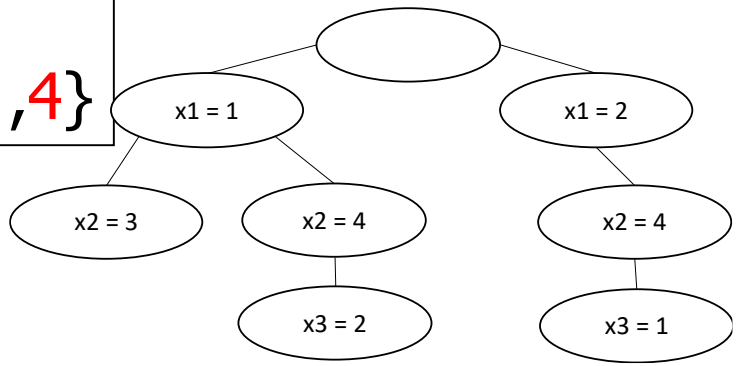




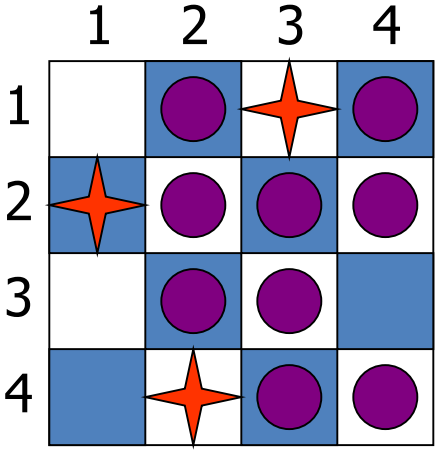
Forward Checking: 4-Queens Problem

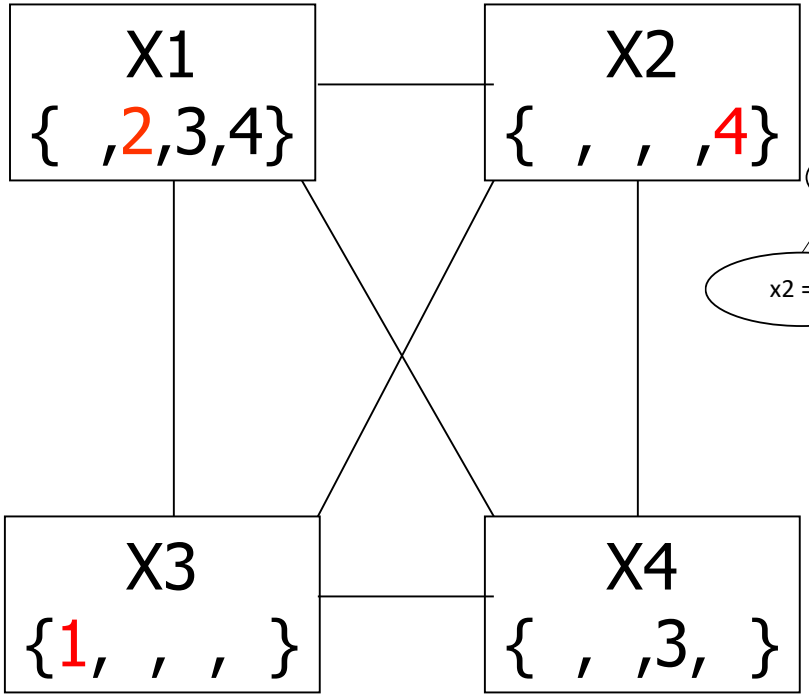


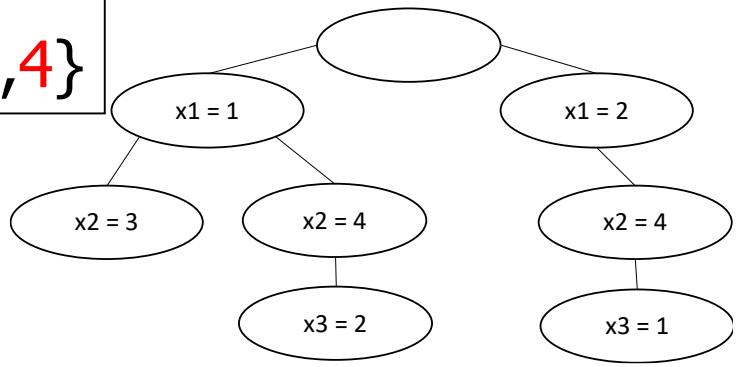




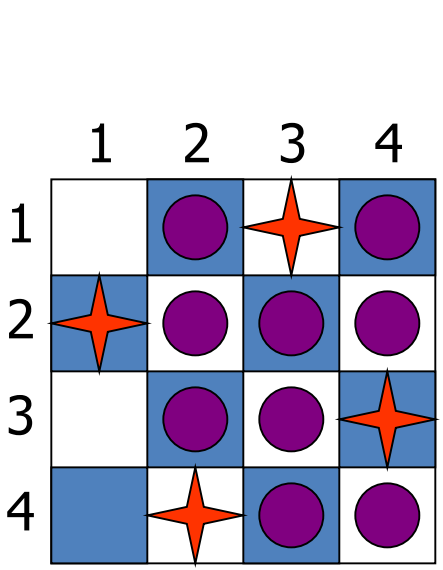
Forward Checking: 4-Queens Problem

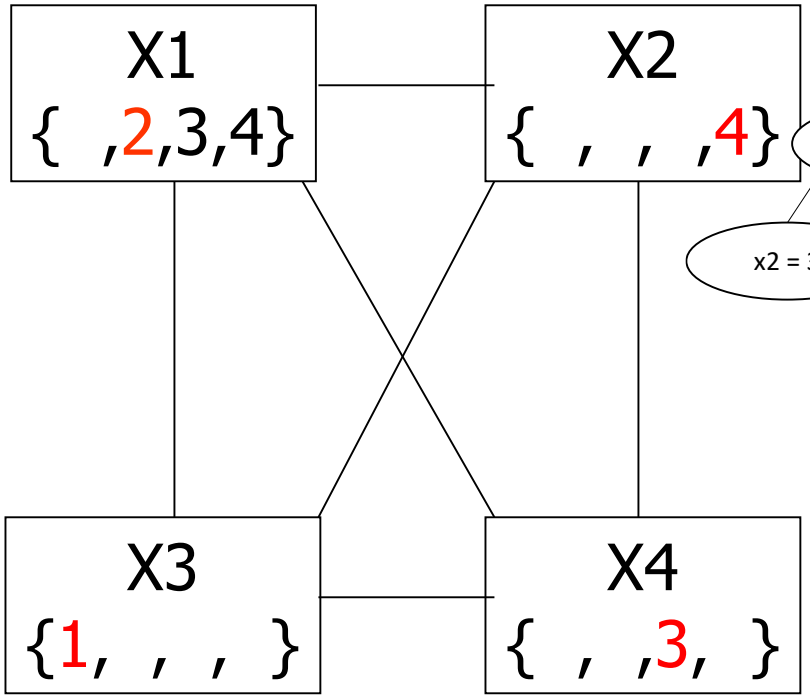


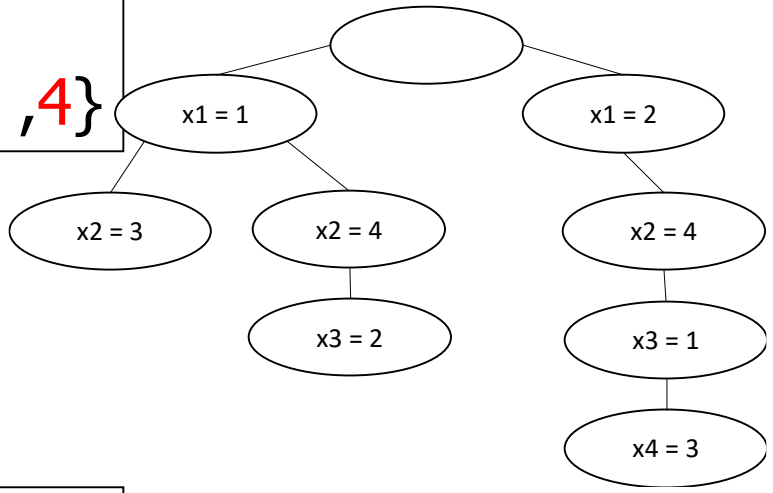




Forward Checking: 4-Queens Problem







Solution !!!!